Document Information	
Filename	TP_F103U-00034_User_Guide_5G_NAD_Module
Status	Release
Revision Date	Jan 21, 2026
Version	18.00
Document ID	TP_F103U-00034

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Table of contents

1 Introduction 7
1.1 Notational Conventions 7
2 Setup 7
2.1 Tools/Equipment Needed 7
2.2 Setup Instructions 7
2.3 Setup Troubleshooting 7
3 Features/Components – Module Interface 8
3.1 Power Up/Down 8
3.1.1 Overview 8
3.1.1.1 Platform Content Overview 8
3.1.1.2 Product Expectations 9
3.1.1.3 Extending Platform Functionality 9
3.1.2 Setup 9
3.1.2.1 Tools/Equipment Needed 9
3.1.2.2 Setup Instructions 9
3.1.2.3 Setup Troubleshooting 9
3.1.3 Usage 9
3.1.3.1 Basic Instructions 9
3.1.3.2 Detailed Instructions 10
3.1.3.3 Testing/Error Reporting 13
3.2 UART 13
3.2.1 Overview 13
3.2.1.1 Platform Content Overview 13
3.2.1.2 Product Expectations 13
3.2.1.3 Extending Platform Functionality 14
3.2.2 Setup 14
3.2.2.1 Tools/Equipment Needed 14
3.2.2.2 Setup Instructions 14
3.2.2.3 Setup Troubleshooting 14
3.2.3 Usage 14
3.2.3.1 Basic Instructions 14
3.2.3.2 Detailed Instructions 14
3.2.3.3 Testing/Error Reporting 14
3.3 USB 14
3.3.1 Overview 14
3.3.1.1 Platform Content Overview 14
3.3.1.2 Product Expectations 14
3.3.1.3 Extending Platform Functionality 14
3.3.2 Setup 14

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.3.2.1 Tools/Equipment Needed 14
3.3.2.2 Setup Instructions 15
3.3.2.3 Setup Troubleshooting 15
3.3.3 Usage 15
3.3.3.1 Basic Instructions 15
3.3.3.2 Detailed Instructions 15
3.3.3.3 Testing/Error Reporting 15
3.4 AT command access via ADB 15
3.4.1 Overview 15
3.4.1.1 Platform Content Overview 15
3.4.1.2 Product Expectations 15
3.4.1.3 Extending Platform Functionality 15
3.4.2 Setup 15
3.4.2.1 Tools/Equipment Needed 15
3.4.2.2 Setup Instructions 16
3.4.2.3 Setup Troubleshooting 16
3.4.3 Usage 16
3.4.3.1 Basic Instructions 16
3.4.3.2 Detailed Instructions 16
3.4.3.3 Testing/Error Reporting 17
3.5 Debug interfaces lock/unlock 17
3.5.1 Overview 17
3.5.1.1 Platform Content Overview 17
3.5.1.2 Product Expectations 17
3.5.1.3 Extending Platform Functionality 18
3.5.2 Setup 18
3.5.2.1 Tools/Equipment Needed 18
3.5.2.2 Setup Instructions 18
3.5.2.3 Setup Troubleshooting 18
3.5.3 Usage 18
3.5.3.1 Basic Instructions 18
3.5.3.2 Detailed Instructions 18
3.5.3.3 Testing/Error Reporting 19
3.6 Double Banking 19
3.6.1 Overview 19
3.6.1.1 Platform Content Overview 19
3.6.1.2 Product Expectations 19
3.6.1.3 Extending Platform Functionality 19
3.6.2 Setup 19
3.6.2.1 Tools/Equipment Needed 19
3.6.2.2 Setup Instructions 20
3.6.2.3 Setup Troubleshooting 20
3.6.3 Usage 20

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.6.3.1 Basic Instructions 20
3.6.3.2 Detailed Instructions 20
3.6.3.3 Testing/Error Reporting 22
3.7 Modem Diagnostic Socket Logging 22
3.7.1 Overview 22
3.7.1.1 Platform Content Overview 22
3.7.1.2 Product Expectations 22
3.7.1.3 Extending Platform Functionality 22
3.7.2 Setup 22
3.7.2.1 Tools/Equipment Needed 22
3.7.2.2 Setup Instructions 22
3.7.2.3 Setup Troubleshooting 25
3.7.3 Usage 25
3.7.3.1 Basic Instructions 25
3.7.3.2 Detailed Instructions 25
3.7.3.3 Automated logging setup 30
3.8 JTAG unlock 32
3.8.1 Overview 32
3.8.1.1 Platform Content Overview 32
3.8.1.2 Product Expectations 32
3.8.1.3 Extending Platform Functionality 32
3.8.2 Setup 32
3.8.2.1 Tools/Equipment Needed 32
3.8.2.2 Setup instructions 32
3.8.2.3 Setup Troubleshooting 32
3.8.3 Usage 32
3.8.3.1 Basic Instructions 32
3.8.3.2 Detailed Instructions 32
3.8.3.3 Testing/Error Reporting 33
4 Features/Components – Application Development 33
4.1 UART 33
4.1.1 Overview 33
4.1.1.1 Platform Content Overview 33
4.1.1.2 Product Expectations 33
4.1.1.3 Extending Platform Functionality 34
4.1.2 Setup 34
4.1.2.1 Tools/Equipment Needed 34
4.1.2.2 Setup Instructions 34
4.1.2.3 Setup Troubleshooting 34
4.1.3 Usage 34
4.1.3.1 Basic Instructions 34
4.1.3.2 Detailed Instructions 34

© Continental Automotive GmbH 2019
Confidential and Proprietary

4.1.3.3 Testing/Error Reporting 35
4.2 Log & Trace 36
4.2.1 Logging: Application usage 36
4.2.1.1 Overview 36
4.2.1.2 Setup 37
4.2.1.3 Usage 37
4.2.2 Logging: Configuring and routing log generation 41
4.2.2.1 Overview 41
4.2.2.2 Setup 42
4.2.2.3 Usage 43
4.3 Secure Watchdog 45
4.3.1 Overview 45
4.3.1.1 Platform Content Overview 45
4.3.1.2 Product Expectations 45
4.3.1.3 Extending Platform Functionality 45
4.3.2 Setup 45
4.3.2.1 Tools/Equipment Needed 45
4.3.2.2 Setup Instructions 45
4.3.2.3 Setup Troubleshooting 45
4.3.3 Usage 45
4.3.3.1 Basic Instructions 45
4.3.3.2 Detailed Instructions 45
4.3.3.3 Testing/Error Reporting 46
4.4 Process Level Software Watchdog 46
4.4.1 Overview 46
4.4.1.1 Platform Content Overview 46
4.4.1.2 Product Expectations 47
4.4.1.3 Extending Platform Functionality 47
4.4.2 Setup 47
4.4.2.1 Tools/Equipment Needed 47
4.4.2.2 Setup Instructions 47
4.4.2.3 Setup Troubleshooting 47
4.4.3 Usage 47
4.4.3.1 Basic Instructions 47
4.4.3.2 Detailed Instructions 47
4.4.3.3 Testing/Error Reporting 48
5 Known Issues 48
6 FAQs 48
7 Platform DocumentationTroubleshooting/Tips and Tricks 48
8 References 48
8.1 Abbreviations/Acronyms/Definitions 48

© Continental Automotive GmbH 2019
Confidential and Proprietary

8.2 Platform Documentation 48
8.3 Website links 49
9 Revision History 50

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides usage information for Continental 5G NAD Module. The Continental 5G NAD Module provides a 5G (5th generation) cellular modem with a Linux based application processor. This document provides detailed usage information that is not included in the 5G NAD Module spec.

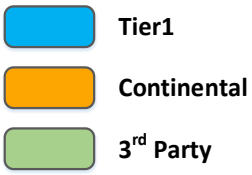
The target audience for this document is developers of components that interface with the NAD Module.

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

2.1 Tools/Equipment Needed

- Continental 5G NAD Module
- 4-volt power supply (see 4V_PMX pin description in 5G NAD Module spec)
- Connection/access to NAD Module GPIO and peripherals of interest
- Continental Software Development Kit (SDK) – for on-target application development

2.2 Setup Instructions

The Continental 5G NAD Module should be physically connected to external peripherals (e.g., personal computer, microcontrollers, discrete components) to facilitate desired access to module functionality. At a minimum, an ability to control the 4-volt power supply and manipulate related power on/off signals (see Power Up/Down) should be supported. Access to other functionality such as UARTs and USB should be provided as applicable.

2.3 Setup Troubleshooting

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

3 Features/Components – Module Interface

This section details external interface to the 5G NAD module. The functionality described in this section would typically be provided by an external processor connected to the module, such as a vehicle bus controller. This is intended for developers working on an external controller for the 5G NAD module.

3.1 Power Up/Down

3.1.1 Overview

3.1.1.1 Platform Content Overview

The following pins on the 5G NAD Module are used for controlling module power up and power down operation:

Pin (Pad)	Pad Name	Description
Y18, Z18	4V_PMX	Input to the NAD module. Main module power supply. Should be connected to a nominal 4-Volt supply capable of providing maximum current specified in the 5G NAD Module spec when the module is on.
V15	RESET_N	Input to the NAD module. Used to request a module reset.[1]
V14	PM_KPD_PWR_N	Input to the NAD module. Used to initiate module power up and power down.
S17	VUC2MDM_HOLDUP	Input to the NAD module. Used to ensure that the 4 volt power supply is provided to the modem chipset within the module.
T21	NAD2PS_HOLDUP	Output from the NAD module. Used within the module to ensure that the 4 volt power supply is provided to the modem chipset within the module. Also used to indicate NAD module power status.

NOTES:

1. Use of RESET_N to reset the NAD module is only recommended as a last resort. RESET_N will initiate a graceful reset of the application processor in the NAD module, but may not initiate a graceful reset of the modem. Recommendation to use alternate communication method (secondary GPIO, serial bus messaging) to request a module reset and only use RESET_N if other reset methods fail.

The following pins on the 5G NAD Module are used for controlling module boot mode during module power up and are unavailable for general use:

Pin (Pad)	Pad Name
G20	BOOT_CFG1
G19	BOOT_CFG2
M14	BT_UART_TX
F14	GPIO_F14
F18	GPIO_F18
V16	FORCED_USB_BOOT

© Continental Automotive GmbH 2019
Confidential and Proprietary

NOTES:

1. **The boot mode configuration pins must not be externally pulled up or pulled down**, as they are read during NAD module boot up.
2. **None of the boot mode configuration pins should be used or configured as a General Purpose input to the NAD module.** They may only be designed and configured as a General Purpose output or their assigned non-GPIO function (e.g., the BT_UART_TX line can be configured as the Bluetooth UART transmit line).
3. Recommend that all boot mode configuration pins are not connected (open) and are not used if possible, to avoid unintended NAD module power up issues.

3.1.1.2 Product Expectations

- Provide connections to control the pins on the NAD module indicated in Power Up/Down Platform Content Overview as inputs to the NAD module.
- Connect a power supply to 4V_PMX input for module operation.
- Optionally provide connection to the pins on the NAD module indicated in Power Up/Down Platform Content Overview as outputs from the NAD module for monitoring module power status.
- Avoid externally connecting or using the boot mode configuration pins (BOOT_CFG1, BOOT_CFG2, BT_UART_TX, GPIO_F14, GPIO_F18, and FORCED_USB_BOOT), i.e., **do not pull up or pull down external to the NAD module**. Only design and configure boot mode configuration pins as a General Purpose output or their assigned non-GPIO function (e.g., the BT_UART_TX line can be configured as the Bluetooth UART transmit line).

3.1.1.3 Extending Platform Functionality

Not Applicable

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not Applicable

3.1.2.2 Setup Instructions

Connect the pins from the NAD module indicated in Power Up/Down Platform Content Overview as inputs to the NAD module to a device that can control each line as indicated. All lines except 4V_PMX are standard digital inputs. 4V_PMX is the power supply input for the module, so must be driven by a power supply with sufficient output power.

Connect the pins from the NAD module indicated in Power Up/Down Overview as outputs from the NAD module to a device that can read the state (value) of each line.

3.1.2.3 Setup Troubleshooting

Not Applicable

3.1.3 Usage

3.1.3.1 Basic Instructions

The PM_KPD_PWR_N pin is used for initiating the module power up and power down sequences.

For module power up, initially RESET_N should be deasserted (e.g., left floating), VUC2MDM_HOLDUP should be asserted (pulled high), and power should be applied to 4V_PMX. Once power is applied, PM_KPD_PWR_N is asserted (pulled low) to initiate the power up sequence. The power management component on the NAD module should initiate power up of the embedded application processor within about 300 milliseconds, but it is

© Continental Automotive GmbH 2019
Confidential and Proprietary

recommended to assert PM_KPD_PWR_N for at least 1 second to ensure power up sequence starts. When power up processing is complete, NAD2PS_HOLDUP will be asserted (pulled high).

For module power down, RESET_N should already be deasserted (e.g., left floating), VUC2MDM_HOLDUP should already be asserted (pulled high), and PM_KPD_PWR_N should already be deasserted (e.g., left floating). To request module power down, PM_KPD_PWR_N is asserted (pulled low) for at least 3 seconds. Then, the module power down sequence is initiated when PM_KPD_PWR_N is deasserted (to ensure the pin assertion is intended). VUC2MDM_HOLDUP should be deasserted prior to PM_KPD_PWR_N deassertion, so recommend to deassert VUC2MDM_HOLDUP when PM_KPD_PWR_N is asserted. When power down processing is complete and power can be safely removed from the module, NAD2PS_HOLDUP will be deasserted (pulled low). After NAD2PS_HOLDUP is deasserted then power can be removed from 4V_PMX.

3.1.3.2 Detailed Instructions

3.1.3.2.1 NAD Module Power Up

The PM_KPD_PWR_N pin needs to be asserted (pulled low) after applying power to the NAD module to initiate module power up sequence. The NAD2PS_HOLDUP pin can be monitored to determine when power up processing is complete and the module is ready to communicate (e.g., UART/USB up).

Table 1: Power Up Timing Parameters and **Error! Reference source not found.** illustrate signal timing for module power up.

Figure 1: Power Up Timing

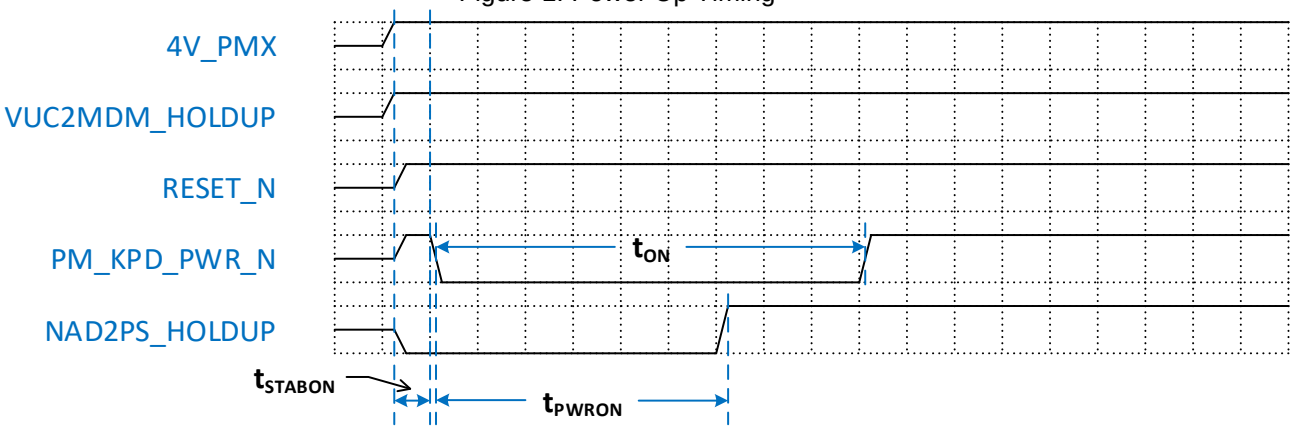


Table 1: Power Up Timing Parameters

Parameter	Description	MIN	TYP	MAX	Units
t_{STABON}	Power applied stabilization (delay before power up pulse)	0.1	-	1	ms
t_{PWRON}	Turn on delay to module boot start	150	200	1000	ms
t_{ON}	Turn on pulse duration	300	-	∞	ms

© Continental Automotive GmbH 2019
Confidential and Proprietary

Note:	PM_KPD_PWR_N must be deasserted (floating due to internal pull-up) prior to module power down. t_{ON} is shown as infinite maximum value because module power up is triggered by the falling edge of PM_KPD_PWR_N. The line only needs to be asserted until NAD2PS_HOLDUP is asserted, so it is recommended to assert the line until about 100 milliseconds after NAD2PS_HOLDUP gets asserted and then deassert instead of leaving the line asserted for an extended period of time.

Example Power Up Sequence

The following example shows a sequence of operations for NAD module power up that may be performed by a controller/processor connected to the NAD module.

1. Deassert RESET_N (e.g., leave floating due to internal pull-up).
2. Apply power to 4V_PMX pin(s).
3. Assert (pull high) VUC2MDM_HOLDUP.
4. Assert (pull low) PM_KPD_PWR_N.
5. Wait for NAD2PS_HOLDUP assertion (goes high).
6. Wait 100 milliseconds.
7. Deassert PM_KPD_PWR_N (e.g., let float due to internal pull-up).

Power Up Failure Processing

The NAD module should power up and start boot up sequence if the power up timing presented in

Table 1: Power Up Timing Parameters is followed. If NAD2PS_HOLDUP is not asserted within the maximum value specified for t_{PWRON} in **Error! Reference source not found.**, then start of module boot up was unsuccessful.

When NAD2PS_HOLDUP assertion is not observed, the recovery steps listed below should be taken to reattempt module power up. If module power up fails 5 times in succession, then there is an issue with connection to the NAD module, NAD module power supply, or an internal NAD module failure.

NAD Module Power Up/Boot Up Recovery Sequence:

1. Deassert (pull low) VUC2MDM_HOLDUP.
2. Remove power from 4V_PMX pin(s).
3. Ensure RESET_N is deasserted (left floating due to internal pull-up).
4. Deassert PM_KPD_PWR_N (e.g., let float due to internal pull-up).
5. Wait 2 seconds.
6. Reattempt module power up sequence.

3.1.3.2.2 NAD Module Power Down

The PM_KPD_PWR_N pin needs to be asserted (pulled low) while the NAD module is running to initiate module power down sequence. This will ensure that a controlled power down of the module is performed. The NAD2PS_HOLDUP pin should be monitored to determine when power down processing is complete and it is safe to remove power. Figure 2 and **Error! Reference source not found.** illustrate signal timing for module power down.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 2: Power Down Timing

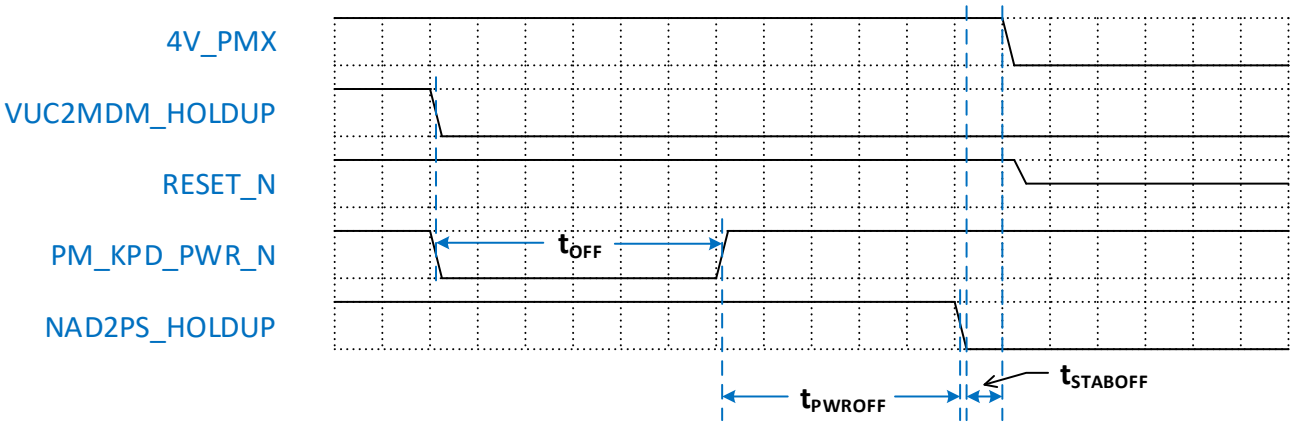


Table 2: Power Down Timing Parameters

Parameter	Description	MIN	TYP	MAX	Units
t_{OFF}	Turn off pulse duration	3	-	7	s
t_{PWROFF}	Turn off delay to power down complete	1	3	10	s
$t_{STABOFF}$	Delay after power down complete before power removal	0.1	-	1	ms

Notes:

1. The NAD module monitors PM_KPD_PWR_N for a transition from deasserted (pulled high internally) to asserted (pulled low) to start a 2-second power down request timer.
2. If PM_KPD_PWR_N is deasserted before the 2-second power down request timer expires, then a power down is not performed. This ensures a valid power down request.
3. Once PM_KPD_PWR_N has been asserted for approximately 2 seconds, the NAD module waits for PM_KPD_PWR_N to be deasserted. Power down processing in the NAD module is only initiated when PM_KPD_PWR_N is deasserted. So it is recommended to assert the line for 3 seconds and then deassert to start module power down processing.
4. VUC2MDM_HOLDUP needs to be deasserted (pull low) some time before power down processing is initiated (e.g., prior to PM_KPD_PWR_N deassertion). Otherwise, the NAD module may reboot instead of powering down. VUC2MDM_HOLDUP can be deasserted when PM_KPD_PWR_N is asserted.
5. VUC2MDM_HOLDUP can be used to abort power down sequence and force a NAD module reset. If VUC2MDM_HOLDUP is asserted when PM_KPD_PWR_N is deasserted, the NAD module will perform a reboot instead of powering down.
6. When the NAD module is on PM_KPD_PWR_N is monitored by systemd. Also, applications can request power down sequencing internally. So, applications need to hold off on any power off request that occurs during boot up and process the request once boot up is complete. It is best that PM_KPD_PWR_N is not asserted again after being deasserted during the module boot up process. A power down initiated during boot up will cause unpredictable results.

Example Power Down Sequence

The following example shows a sequence of operations for NAD module power down that may be performed by a controller/processor connected to the NAD module. This example assumes that PM_KPD_PWR_N is deasserted, RESET_N is deasserted, and the NAD module is awake (i.e., not suspended) at step 1.

1. Assert (pull low) PM_KPD_PWR_N.
2. Deassert (pull low) VUC2MDM_HOLDUP.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3. Wait 3 seconds.
4. Deassert PM_KPD_PWR_N (e.g., let float due to internal pull-up). Note that NAD module power down is initiated at this point.
5. Wait for NAD2PS_HOLDUP deassertion (goes low).
6. Remove power from 4V_PMX pin(s).

Power Down Failure Processing

The NAD module should power down if the power down timing presented in Figure 2 is followed. If NAD2PS_HOLDUP remains asserted after the maximum value specified for t_{PWROFF} in **Error! Reference source not found.** after PM_KPD_PWR_N is deasserted (let float due to internal pull-up), then a controlled power down of the module was unsuccessful, and module power down should be forced.

In order to account for unsuccessful controlled power down of the NAD module it is recommended to start a fail-safe timer when PM_KPD_PWR_N is deasserted to initiate the power down process. The timeout for the fail-safe timer should be sufficiently larger than t_{PWROFF} maximum value, but short enough to prevent an extended power down delay (e.g., 15 seconds). If a timeout of the fail-safe timer occurs, then power should be removed from 4V_PMX pins(s) regardless of NAD2PS_HOLDUP status.

3.1.3.3 Testing/Error Reporting

Not Applicable

3.2 UART

3.2.1 Overview

3.2.1.1 Platform Content Overview

The 5G NAD Module supports the following UARTs:

UART name	Pins (Pads)	Pad Names	Description
IPC UART	A20 B20 A19 B19	IPC_UART_CTS IPC_UART_RTS IPC_UART_RX IPC_UART_TX	Inter-processor communication channel. Used for message exchange between the Cortex A7 application processor on the NAD module and an external controller/processor (e.g., vehicle interface module).
NAD debug UART	C19 C20	DBG_UART_RX DBG_UART_TX	Debug UART. Linux console.
BT UART	M15 N15 N14 M14	BT_UART_CTS BT_UART_RTS BT_UART_RX BT_UART_TX	Bluetooth UART. Used for communication with a Bluetooth transceiver.

3.2.1.2 Product Expectations

- Provide connections to pins associated with desired UART functionality as indicated in UART Platform Content Overview.
    - Typically, the IPC UART is connected directly to a vehicle interface microcontroller for message exchange. Note that all 4 pins of the IPC UART need to be connected/used due to default UART configuration settings.
    - Typically, the NAD debug UART is made available to a Personal Computer (PC) or similar device for module debugging via Linux console.
    - Typically, the BT UART is connected directly to a Bluetooth transceiver for Bluetooth communication.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.1.3 Extending Platform Functionality

Not Applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not Applicable

3.2.2.2 Setup Instructions

Not Applicable

3.2.2.3 Setup Troubleshooting

Not Applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

Ensure that UARTs from the NAD module are connected to devices capable of supporting the desired baud rate, data bits, parity, and stop bits.

3.2.3.2 Detailed Instructions

Not Applicable

3.2.3.3 Testing/Error Reporting

Not Applicable

3.3 USB

3.3.1 Overview

3.3.1.1 Platform Content Overview

The 5G NAD Module supports a High-Speed (HS) USB for access to Android Debug Bridge (ADB). The HS USB is realized via the following pins on the module:

- Pad J24: USB_HS_DM
- Pad K24: USB_HS_DP
- Pad K22: USB_ID
- Pad J22: USB_VBUS_DET

3.3.1.2 Product Expectations

- Provide connections to pins associated with HS USB functionality as indicated in USB Platform Content Overview.

3.3.1.3 Extending Platform Functionality

Not Applicable

3.3.2 Setup

3.3.2.1 Tools/Equipment Needed

- Personal Computer (PC) with Windows 10 operating system

© Continental Automotive GmbH 2019
Confidential and Proprietary

- winusb USB Host Driver (native on Windows 10)
- High Speed USB cable

3.3.2.2 Setup Instructions

- Install ADB following the instructions on https://developer.android.com/studio/command-line/adb on PC.

3.3.2.3 Setup Troubleshooting

Not Applicable

3.3.3 Usage

3.3.3.1 Basic Instructions

Once ADB has been installed on the Personal Computer (PC) and the NAD module HS USB port is attached via a High Speed USB cable, a Windows terminal (e.g., cmd Window) can be used to access ADB. Note that ADB should be accessible with the winusb driver that is included with Windows 10.

Note: **ADB is only available during development when the test partition (`/tst`) is mounted**. The test partition should not be available in production due to security issues. Thus, features/functionality dependent on ADB (e.g., fastboot, AT command access) are not available in production.

3.3.3.2 Detailed Instructions

See https://developer.android.com/studio/command-line/adb for information on ADB.

3.3.3.3 Testing/Error Reporting

Not Applicable

3.4 AT command access via ADB

3.4.1 Overview

3.4.1.1 Platform Content Overview

AT commands can be sent to the modem in the 5G NAD module via ADB message tunneling. Modem access via AT commands may be useful during development for testing basic modem functionality. Note that using AT commands could interfere with application access to the modem.

3.4.1.2 Product Expectations

- Install ADB (see https://developer.android.com/studio/command-line/adb) on Windows 10 Personal Computer (PC).
- Connect the NAD module HS USB port to the NAD module via a High Speed USB cable as specified in section 3.3.

3.4.1.3 Extending Platform Functionality

Not Applicable

3.4.2 Setup

3.4.2.1 Tools/Equipment Needed

- Personal Computer (PC) with Windows 10 operating system
- winusb USB Host Driver (native on Windows 10)
- High Speed USB cable

© Continental Automotive GmbH 2019
Confidential and Proprietary

- ADB (see https://developer.android.com/studio/command-line/adb)

3.4.2.2 Setup Instructions

1. Install ADB following the instructions on https://developer.android.com/studio/command-line/adb on PC.

3.4.2.3 Setup Troubleshooting

Not Applicable

3.4.3 Usage

3.4.3.1 Basic Instructions

Not Applicable

3.4.3.2 Detailed Instructions

Note: Illustrated commands are performed within an open Windows terminal (e.g., cmd Window).

1. Open adb shell

```
>adb shell
```

2. Redirect shared memory channel to console

```
[root@sa515m ]#cat /dev/at_mdm0 &
```

3. Send AT command(s)

```
[root@sa515m ]#echo -e "<AT_command>\r\n" > /dev/at_mdm0
```

Where "*<AT_command>*" is the desired AT command.

4. Exit adb shell

```
[root@sa515m ]#exit
```

Example for 'ATI' and 'AT+CFUN?' commands (prompts are yellow, input commands are blue, output/response is white):

```
>adb shell
#################################################################
# Copyright (C) 2019 Continental Automotive Systems, Inc.
#
# Chestnut Development Board
#
# Platform Version: OTP-SA515M-3.1.2.8-DEVEL
# Linux Version:    4.14.111LE.UM.4.1.1-03000-SDX55-51-gd28d138
# NAND info:        2680AC2C 512M/256K/4K/8b
# RAM info:         96908K /291068K available
# Build Date:       Sat Aug 17 08:43:52 UTC 2019
#
# WARNING! Development: Console always allowed
#           (DO NOT PROVIDE IMAGE TO OEM!)
#################################################################
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
[root@sa515m ]#cat /dev/at_mdm0 &
cat /dev/at_mdm0 &
[root@sa515m ]#echo -e "ATI\r\n" > /dev/at_mdm0
echo -e "ATI\r\n" > /dev/at_mdm0
ATI
Manufacturer: QUALCOMM INCORPORATED
Model: 334
Revision: MPSS.HI.1.0.r17-00005-SDX55_GEN_PACK-1.215263.10 1 [Jul 31 2019
16:00:00]
SVN: 00
IMEI:
+GCAP: +CGSM,+DS

OK
[root@sa515m ]#echo -e "AT+CFUN?\r\n" > /dev/at_mdm0
echo -e "AT+CFUN?\r\n" > /dev/at_mdm0
[root@sa515m ]#AT+CFUN?
+CFUN: 7

OK
[root@sa515m ]#exit
```

3.4.3.3 Testing/Error Reporting

Not Applicable

3.5 Debug interfaces lock/unlock

3.5.1 Overview

3.5.1.1 Platform Content Overview

The platform provides the possibility to lock debug interfaces at runtime by blowing the End of line production security fuse using a provided utility.

Debug interfaces can be unlocked on commercial secure devices with the help of debug policy image.

3.5.1.2 Product Expectations

At the time of EOL OEMs must blow the production security fuse to prevent access to the target through debug interfaces.

This action is essential to ensure that production security cannot be bypassed.

Debug policy images can be used to enable debug interfaces on production security enabled devices. `/.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml` file is used to configure, validate, and generate debug policy image (`apdp.mbn`), which enables debugging of images on target.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.5.1.3 Extending Platform Functionality

Not Applicable

3.5.2 Setup

3.5.2.1 Tools/Equipment Needed

- Personal Computer (PC) with Windows 10 operating system
- winusb USB Host Driver (native on Windows 10)
- High Speed USB cable
- ADB (see https://developer.android.com/studio/command-line/adb)

3.5.2.2 Setup Instructions

Install ADB following the instructions on https://developer.android.com/studio/command-line/adb on PC.

3.5.2.3 Setup Troubleshooting

Not Applicable

3.5.3 Usage

3.5.3.1 Basic Instructions

Not Applicable

3.5.3.2 Detailed Instructions

3.5.3.2.1 Lock debug interfaces

To enable End of line production security on a device and lock debug interfaces execute the following utility that is provided with a test image:*/tst/bin/cas-enable-prod-security*
This will blow a corresponding production security hardware fuse. Fastboot will be locked on the next reboot cycle.

3.5.3.2.2 Unlock debug interfaces

To unlock debug interfaces on a production security enabled device, flash an appropriately configured debug policy image on to the target:

1. Update a debug policy configuration file */.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml* by:
   a. Set `DP_ENABLE_PROD_EOL_FUSE_OVERRIDE_OEM` flag to 1:

```
        <flag> <bit_pos>60</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_SUSPEND_SECUREBOOT_OEM-->
        <flag> <bit_pos>61</bit_pos> <value>1</value> </flag> <!--DP_ENABLE_PROD_EOL_FUSE_OVERRIDE_OEM-->
    </flags>
```

   `DP_ENABLE_PROD_EOL_FUSE_OVERRIDE_OEM` flag:
   · 0: (default): EOL fuse is not overridden
   · 1: override production EOL fuse value to unlock debug interfaces on a production security enabled device

   b. Add a device serial number need to be added to the list:

```
    <serial_num_list> <!--Range: 1 to 200-->
        <serial_num>0x2466e4ca</serial_num>
    </serial_num_list>
```

   It is possible to specify up to 200 unique serial numbers in this list field. All of the corresponding devices are then supported by the debug policy and subsystem ramdump will work.

© Continental Automotive GmbH 2019
Confidential and Proprietary

2. Make a debug policy `apdp.mbn` image (see section "Suspend Secure Boot" of 5G NAD SecureBoot User Guide - TP_F101U-00050).

3. Copy signed debug policy image `apdp.mbn` from *`/release/images/devel/debugpolicy/sdx55/dbgp_ap/`* to *`/release/images/devel/4K/`*

4. Add following line to the file *`/release/images/devel/4K/rawprogram_nand_p4K_b256K.xml`*:
```
<program PAGES_PER_BLOCK="64" SECTOR_SIZE_IN_BYTES="4096" filename="apdp.mbn"
label="apdp"          last_sector="7423"          num_partition_sectors="128"
physical_partition_number="0"                           start_sector="7296"/>
```

   If the line is already in the file, modify it by setting `apdp.mbn` to the filename field.

5. Enable EDL mode on the target and flash the image from 4K folder using the supplier flashing tools.

6. (Optional) In order to unlock ADB shell, test partition should be available. Flash the test partition via fastboot using next commands (`test.ubifs` file is in 4K folder):
```
fastboot flash test <test.ubifs file with path>
fastboot reboot
```

3.5.3.3 Testing/Error Reporting

Not Applicable

3.6 Double Banking

3.6.1 Overview

3.6.1.1 Platform Content Overview

Platform provides double banking capabilities which basically contains two set of images for specific components within the NAD layout allowing to boot from one set or another. These sets are known as Active and Inactive banks.

- Active Bank: From where the system is booting.
- Inactive Bank: Copy of the Active bank that is not being used.

Also, platform provides an `tpHAL_getActiveBank` API to query from which bank the system booted

3.6.1.2 Product Expectations

The following expectations must be met by products (users of the platform) for this feature to work as designed:

- Select the appropriate bank for software execution during boot up based on business logic.
- Select the bank requested by software loading during software updates.

3.6.1.3 Extending Platform Functionality

Not Applicable

3.6.2 Setup

3.6.2.1 Tools/Equipment Needed

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.6.2.2 Setup Instructions

Not Applicable

3.6.2.3 Setup Troubleshooting

Not Applicable

3.6.3 Usage

3.6.3.1 Basic Instructions

Drive `VUC2NAD_BANK_SELECT` GPIO pin to change the bank from where the system is booting. Refer to 5G NAD Module spec for further information about bank selection based on the GPIO value.

Use `tpHAL_getActiveBank` API to query from which bank the system has booted.

The GPIO name is `vuc2nad-bank-select-gpio` and state can be queried by HAL API. Refer to TP_F101U-00039 for further information.

Make sure that all these image files are considered during the update process. All these image files must be present on target and all be from the same release version when the update is complete:

Image file
aop.mbn
tz.mbn
devcfg.mbn
hyp.mbn
abl.elf
uefi.elf
<sbl file>.mbn
boot.img
system.ubifs
dsp2.ubifs
multi_image.mbn

3.6.3.2 Detailed Instructions

3.6.3.2.1 VUC2NAD_BANK_SELECT GPIO configuration within update process:

- The GPIO should be set to the desired value to the appropriate bank before a reset is triggered after update status signal with E_SWL_STATE_UPD_OK_REBOOT_REQUIRED is reported.

3.6.3.2.2 How to override GPIO bank select

This section contains instructions to override the GPIO bank selection.

Embedded in the document it is override_gpIo_boot.zip file

override_gplo_boot.zip

© Continental Automotive GmbH 2019
Confidential and Proprietary

which contains

- override_gpIo_boot_from_primary_bank: used to indicate the NAD to boot from primary bank.
- override_gpIo_boot_from_secondary_bank: used to indicate the NAD to boot from secondary bank.

In order to instruct the NAD to boot from primary bank

- use adb to push override_gpIo_boot_from_primary_bank into /data folder

```
[root@sa515m ]# adb push override_gpIo_boot_from_primary_bank /data
```

- In the target from command line run

```
[root@sa515m ]# flash_eraseall /dev/mtd6
[root@sa515m ]# cat /data/override_gpIo_boot_from_primary_bank > /dev/mtd6
```

In order to instruct the NAD to boot from secondary bank

- use adb to push override_gpIo_boot_from_secondary_bank into /data folder

```
[root@sa515m ]# adb push override_gpIo_boot_from_secondary_bank /data
```

- In the target from command line run

```
[root@sa515m ]# flash_eraseall /dev/mtd6
[root@sa515m ]# cat /data/override_gpIo_boot_from_secondary_bank > /dev/mtd6
```

In order to clear the override

- In the target from command line run

```
[root@sa515m ]# flash_eraseall /dev/mtd6
```

3.6.3.2.3 tpHAL_getActiveBank

This section contains HAL interface APIs and types relating to active bank selection. More details can be found in Doxygen with the following format unless stated otherwise:

Doxygen Path
|-> ***Name of Data Type or API***

Where *Doxygen Path* is

```
Modules
   |-> Hardware Abstraction Layer (HAL)
      |-> HAL Library
         |-> System Interface
```

API/Type Description	API/Type Name as Referenced in Doxygen Path
Enumeration of HAL API return codes	**Modules** **|-> Hardware Abstraction Layer (HAL)** **|-> HAL Library** **|-> tpHAL_return_t**

© Continental Automotive GmbH 2019
Confidential and Proprietary

API/Type Description	API/Type Name as Referenced in Doxygen Path
Identifies which executable bank, of the two banks in the system, is currently active.	`tpHAL_getActiveBank`
Enumeration of the active bank.	`tpHAL_BankSelect_t`

In case `tpHAL_getActiveBank` API is not used. The active bank value can be query by reading the contents of `/proc/board/bank_selection` Linux procfs entry. See table below to map the reported value with bank selection:

Bank selection	Bank
0	`Primary Bank is Active.`
1	`Secondary Bank is Active.`

3.6.3.3 Testing/Error Reporting

Not Applicable

3.7 Modem Diagnostic Socket Logging

3.7.1 Overview

3.7.1.1 Platform Content Overview

Telematics platform provides diagnostic traffic over Ethernet if the device has Ethernet enabled. The modem diagnostic logs could be routed through TCP sockets through Ethernet port to external storage device. The remote end could be a log collector which receives and stores the diagnostic traffic. NAD role can be set as client or server depending on user configuration. Diagnostic traffic over ethernet will be stopped with the system suspend and resumed with the system resume.

3.7.1.2 Product Expectations

- Install ADB (see https://developer.android.com/studio/command-line/adb) on Windows Personal Computer (PC). For automated logging tool ADB version >1.0.31 required.
- Connect the NAD module Ethernet port to the PC and bring up the Ethernet interface as detailed in instructions (section Setup Instructions).

3.7.1.3 Extending Platform Functionality

Not Applicable

3.7.2 Setup

3.7.2.1 Tools/Equipment Needed

Not Applicable

3.7.2.2 Setup Instructions

1) Bring Ethernet interface up on target board.

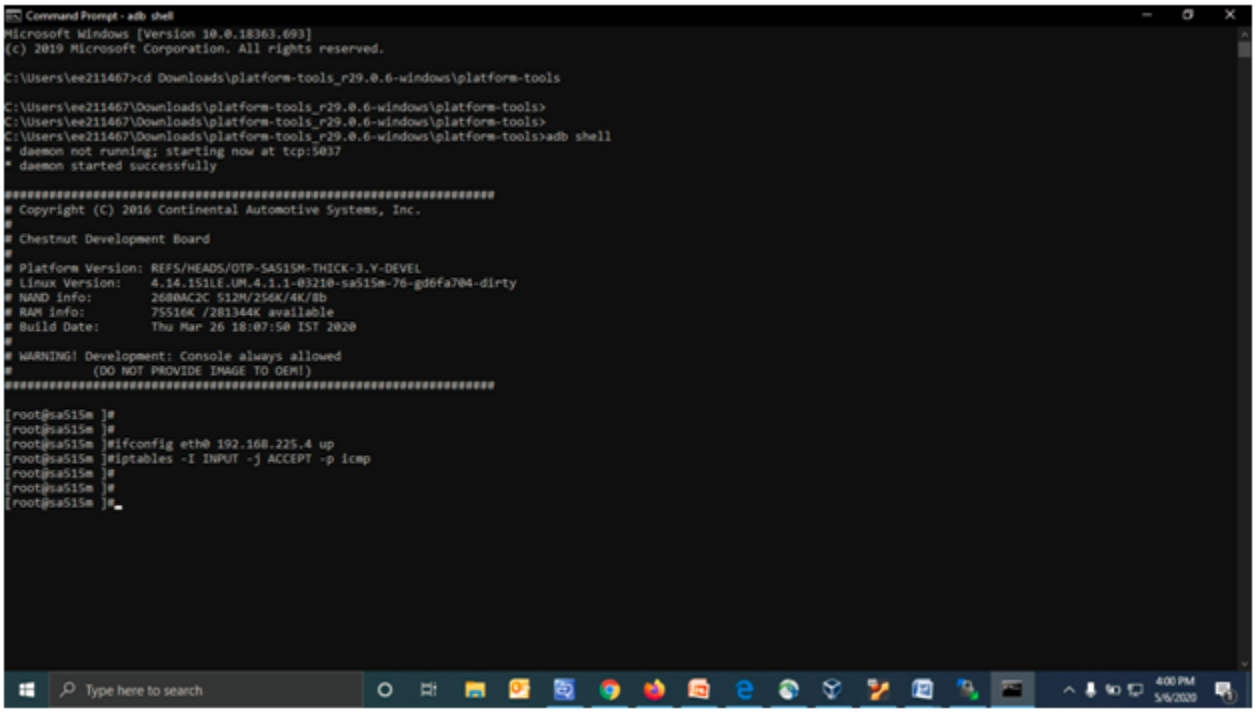
2) Platform should be able to route diagnostic packets using TCP sockets through the Ethernet interface. Change the firewall rules as required.

The following example illustrates setting up an Ethernet connection with a TCP server or client on a PC:

I. Execute command “ifconfig eth0 <*server/client IP*> up” from adb shell of target, where <*server IP*> is the TCP server IP address (screenshots show command using a value of 192.168.225.4).

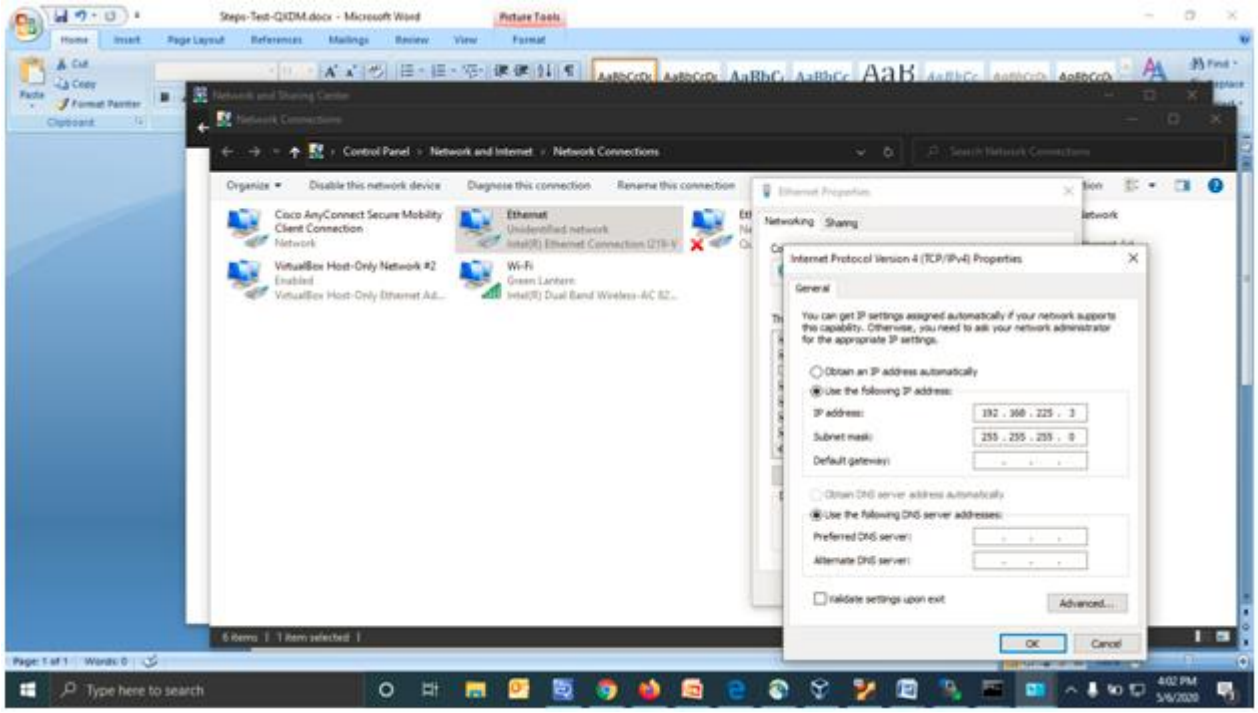
© Continental Automotive GmbH 2019
Confidential and Proprietary

II. Execute command “iptables -I INPUT -j ACCEPT -p icmp”



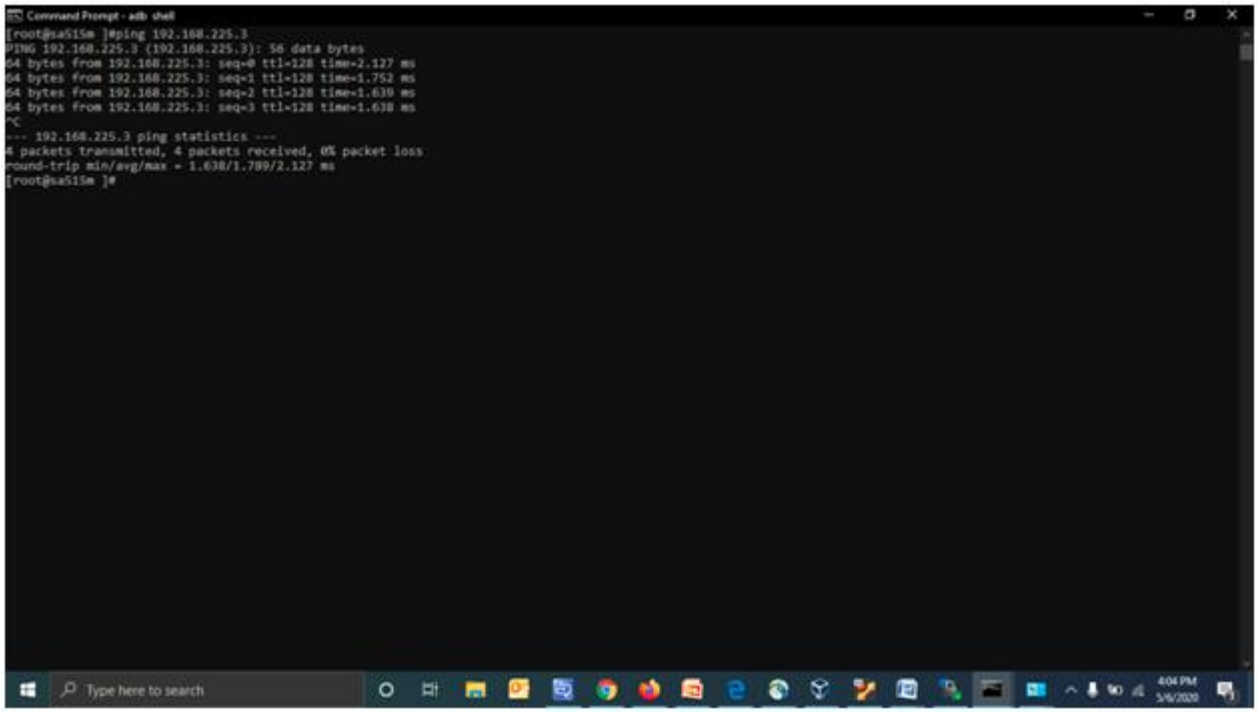
III. Connect Ethernet port of target to PC

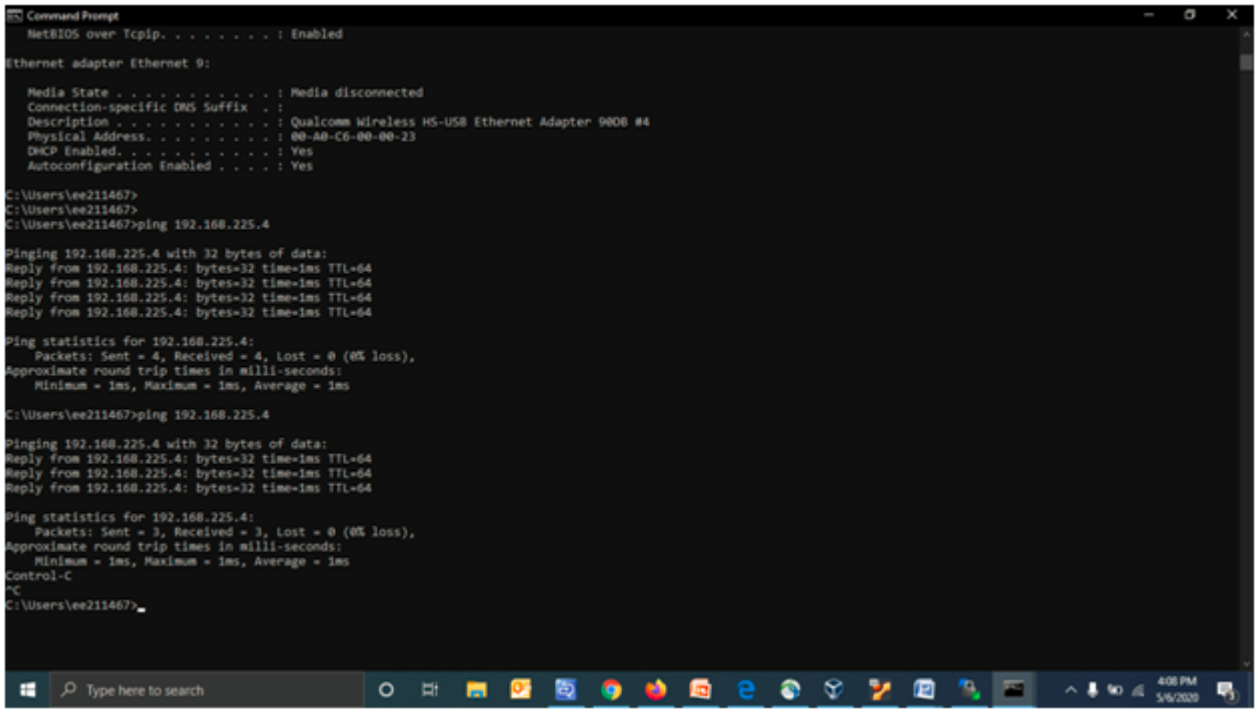
IV. Assign IP address to Ethernet adapter of PC. An IP address of 192.168.225.3 is used in this example.



© Continental Automotive GmbH 2019
Confidential and Proprietary

V. Ping target from PC and vice versa





Note:- The feature requires Ethernet physical link to be connected throughout socket logging. If physical link is disconnected when socket logging is on-going the next cycle of socket logging might exhibit undefined behavior.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.7.2.3 Setup Troubleshooting

Not Applicable

3.7.3 Usage

3.7.3.1 Basic Instructions

Not Applicable

3.7.3.2 Detailed Instructions

1) When NAD is server, the remote storage device which would receive the diagnostic logs routed through Ethernet interface of target board, should run a TCP client. NAD listens to connections from remote storage device. Embedded here is an example TCP client code.

   diag_socket_logger_example.c

   The example C code accepts server IP address (IP address of NAD) and port as input to create a client socket and then read the mask config file. It will then parse the mask file content by delimiter 7e and send each command over socket. The socket logging will start after receiving the commands and then the received packets over socket is dumped in test.qmdl file in logger.

2) When NAD is client, the remote storage device which would receive the diagnostic logs routed through Ethernet interface of target board, should run a TCP server which listens to connections from Telematics device. Embedded here is an example TCP server script.

   diag_server_set_mask_example.pl

3) To enable diag packet masks with on-device logging, encoded mask file needs to be parsed for command and send over socket to NAD. Inside the mask file commands are separated using “7e” delimiters. The two bytes just before “7e” is CRC. The rest of the bytes in between the two “7e” is command.

   Reference hexdump of the mask file (first line)

   0000000 1c1d 7e3b 7800 7ef0 937c 7e49 951c 7e2a

   Command interpreted.

   1d 1c 3b 7e // Command = 1d, CRC = 1c 3b,
   00 78 f0 7e // Command = 00
   7c 93 49 7e // Command = 7c
   1c 95 2a 7e // Command = 1c

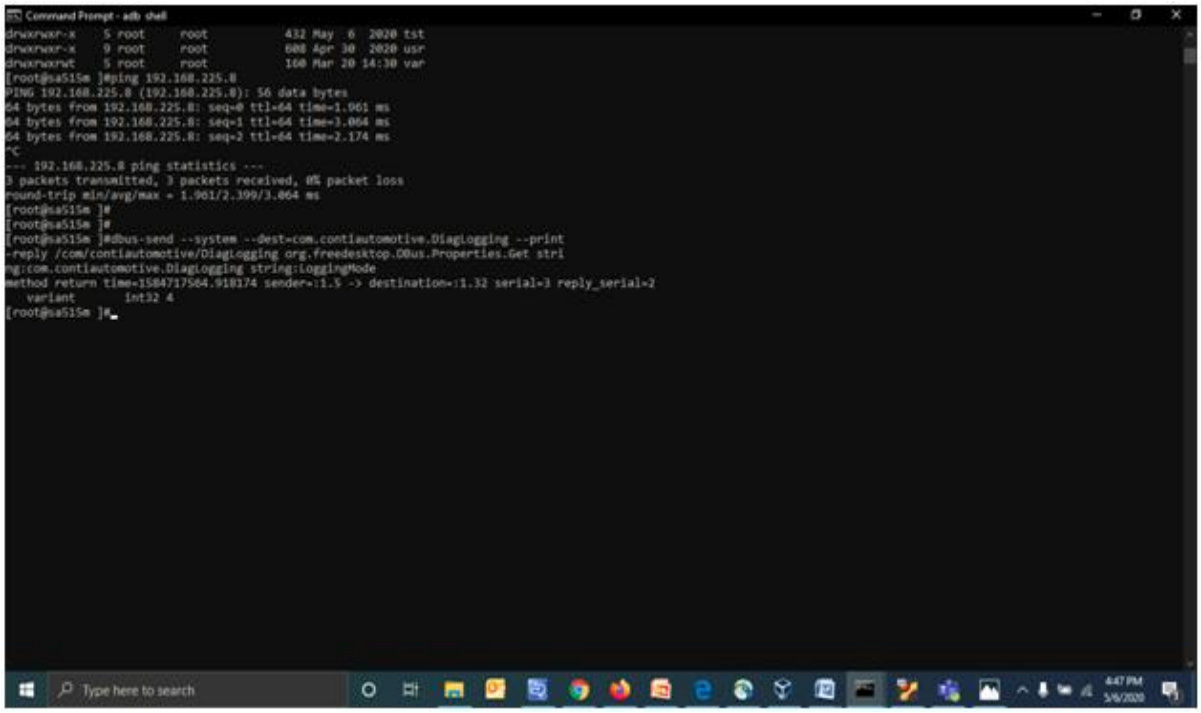
   Logger example sample code has the same logic which reads the bytes from mask file and parse the commands by delimiter and send each command by maintaining the above format.

4) Execute command "sudo -u modem dbus-send --system --dest=com.contiautomotive.DiagLogging --print-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Get string:com.contiautomotive.DiagLogging string:LoggingMode" on target

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Get
                    string:com.contiautomotive.DiagLogging string:LoggingMode
method return time=1597365702.938320 sender=:1.37 -> destination=:1.46 serial=11 reply_serial=2
   variant       int32 4
```

5) Confirm the result is 4 (logging mode - disabled)



6) Execute command "sudo -u modem dbus-send --system --dest=com.contiautomotive.DiagLogging --print-reply /com/contiautomotive/DiagLogging com.contiautomotive.DiagLogging.EnableSocketLogging string:<*server IP*> int32:<*port*> int32:<*role*>" on target

Note: <server IP> is the IP address of the TCP server target will be connected to. <port> is the port number for the Ethernet connection. <role> is the role of NAD (0: client, 1: server)

```
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging com.contiautomotive.DiagLogging.Enable
SocketLogging string:192.168.225.8 int32:2500 int32:0
method return time=1597364735.412828 sender=:1.37 -> destination=:1.44 serial=9 reply_serial=2
   int32 0
```

7) When NAD is configured as client, TCP server listening on the ip address and port as in above command should send a mask file over the socket connection to the client. Please refer the example TCP server example c file under step 1 of Detailed Instructions. Embedded below is an example mask file to be sent through socket connection to client.

on_device_logging_mask.cfg

Please refer to step 3 for information about the sending procedure of commands over socket.

© Continental Automotive GmbH 2019
Confidential and Proprietary

(Alternative to this is to execute command "sudo -u modem dbus−send −−system −−dest=com.contiautomotive.DiagLogging −−print−reply /com/contiautomotive/DiagLogging com.contiautomotive.DiagLogging.ReadMaskFile string:"""" on the target to set default mask. The mask should be stored at /data/on_device_logging/)

8) When NAD is configured as server, NAD will listen on the ip address and port as in step 4. Logger send a mask file over the socket connection to NAD.

9) Logger should receive the packets of data sent by NAD and dump the binary data into a .qmdl file

10) Execute command "sudo -u modem dbus-send --system --dest=com.contiautomotive.DiagLogging --print-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Set string:com.contiautomotive.DiagLogging string:LoggingMode variant:int32:4" to disable socket logging on target

```
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Set st
ring:com.contiautomotive.DiagLogging string:LoggingMode variant:int32:4
method return time=1597365558.968174 sender=:1.37 -> destination=:1.45 serial=10 reply_serial=2
```

11) Execute command "sudo -u modem dbus-send --system --dest=com.contiautomotive.DiagLogging --print-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Get string:com.contiautomotive.DiagLogging string:LoggingMode" on target and the logging mode read back should be 4 (logging disabled)

```
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Get
                 string:com.contiautomotive.DiagLogging string:LoggingMode
method return time=1597364727.831338 sender=:1.37 -> destination=:1.43 serial=8 reply_serial=2
   variant       int32 4
```

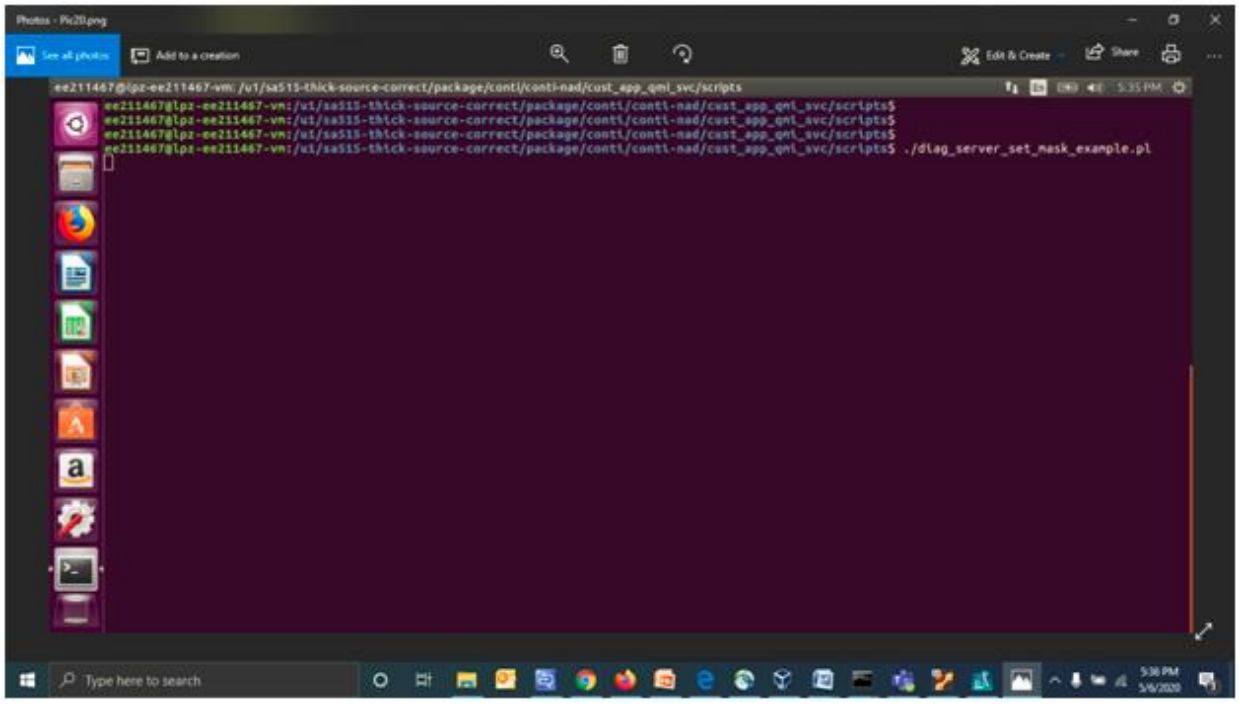
```
64 bytes from 192.168.225.8: seq=26 ttl=64 time=2.001 ms
64 bytes from 192.168.225.8: seq=27 ttl=64 time=1.921 ms
64 bytes from 192.168.225.8: seq=28 ttl=64 time=1.839 ms

--- 192.168.225.8 ping statistics ---
29 packets transmitted, 29 packets received, 0% packet loss
round-trip min/avg/max = 1.647/2.067/8.129 ms
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Get st
ring:com.contiautomotive.DiagLogging string:LoggingMode
method return time=1597361838.573122 sender=:1.6 -> destination=:1.23 serial=3 reply_serial=2
   variant       int32 4
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging com.contiautomotive.DiagLogging.Enable
SocketLogging string:192.168.225.8 int32:2500 int32:0
method return time=1597361865.982209 sender=:1.6 -> destination=:1.24 serial=4 reply_serial=2
   int32 0
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Set st
ring:com.contiautomotive.DiagLogging string:LoggingMode variant:int32:4
method return time=1597362021.096207 sender=:1.6 -> destination=:1.25 serial=5 reply_serial=2
[root@sa515m ~]# dbus-send --system --dest=com.contiautomotive.DiagLogging --pri
nt-reply /com/contiautomotive/DiagLogging org.freedesktop.DBus.Properties.Get st
ring:com.contiautomotive.DiagLogging string:LoggingMode
method return time=1597362031.378189 sender=:1.6 -> destination=:1.26 serial=6 reply_serial=2
   variant       int32 4
```

The following example illustrates modem socket diagnostic packet logging via an Ethernet connection to a TCP server on a PC:

1) TCP server is run on PC to accept client connection requests on a particular port

© Continental Automotive GmbH 2019
Confidential and Proprietary

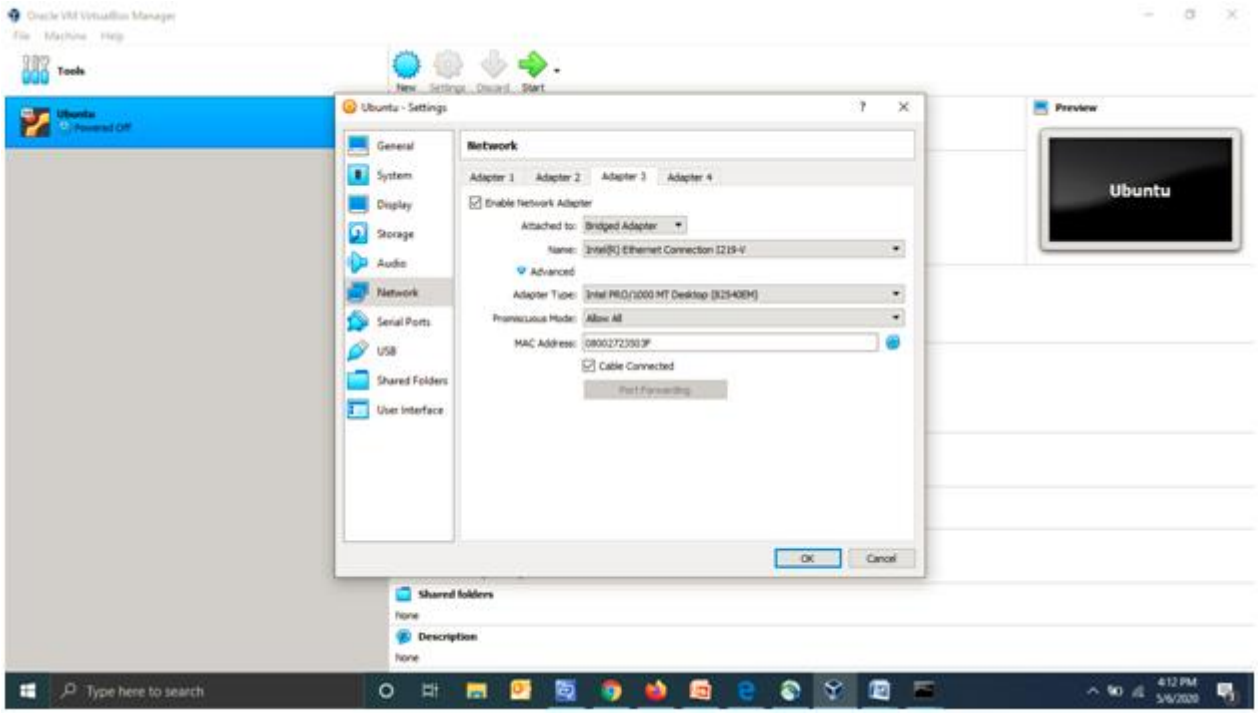
(dbus commands used to enable socket logging on the target can configure ip address and port where server will be listening)



If the server is to run on virtual machine the network settings of the VM should use a bridge adapter to expose the ip address of virtual machine.

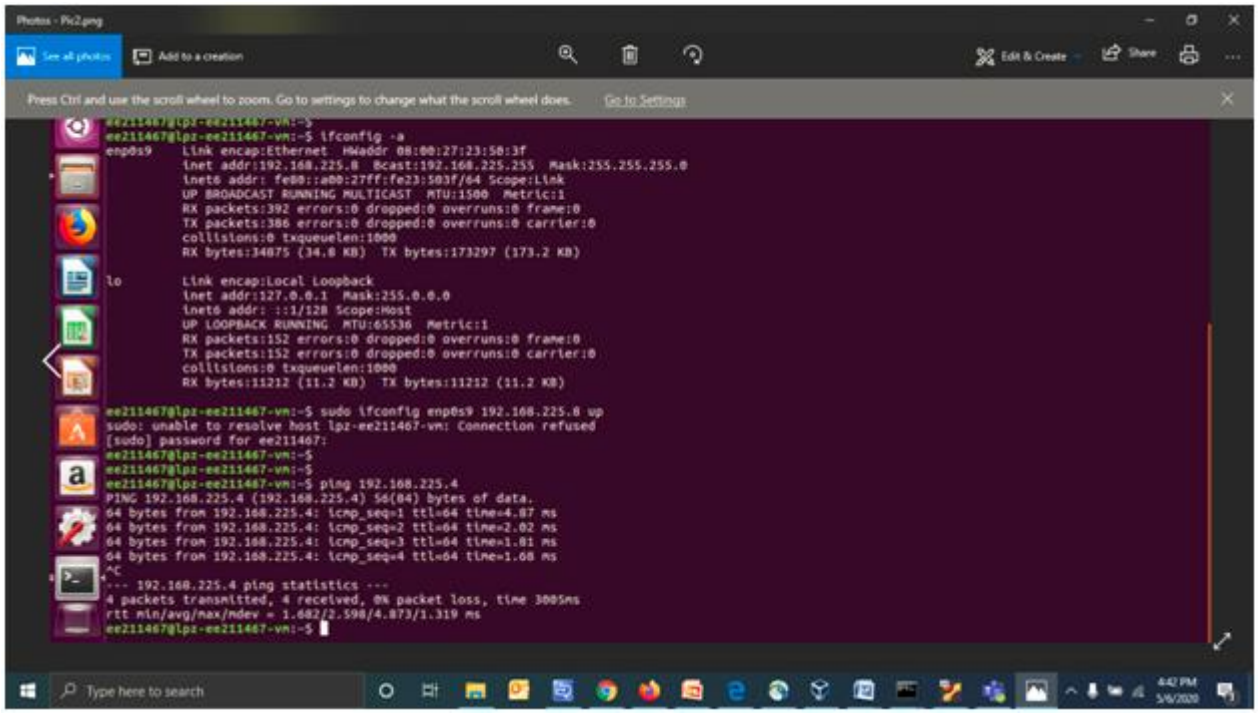
i. Bridge adapter should choose the Ethernet adapter of windows machine in network settings of virtual box

© Continental Automotive GmbH 2019
Confidential and Proprietary

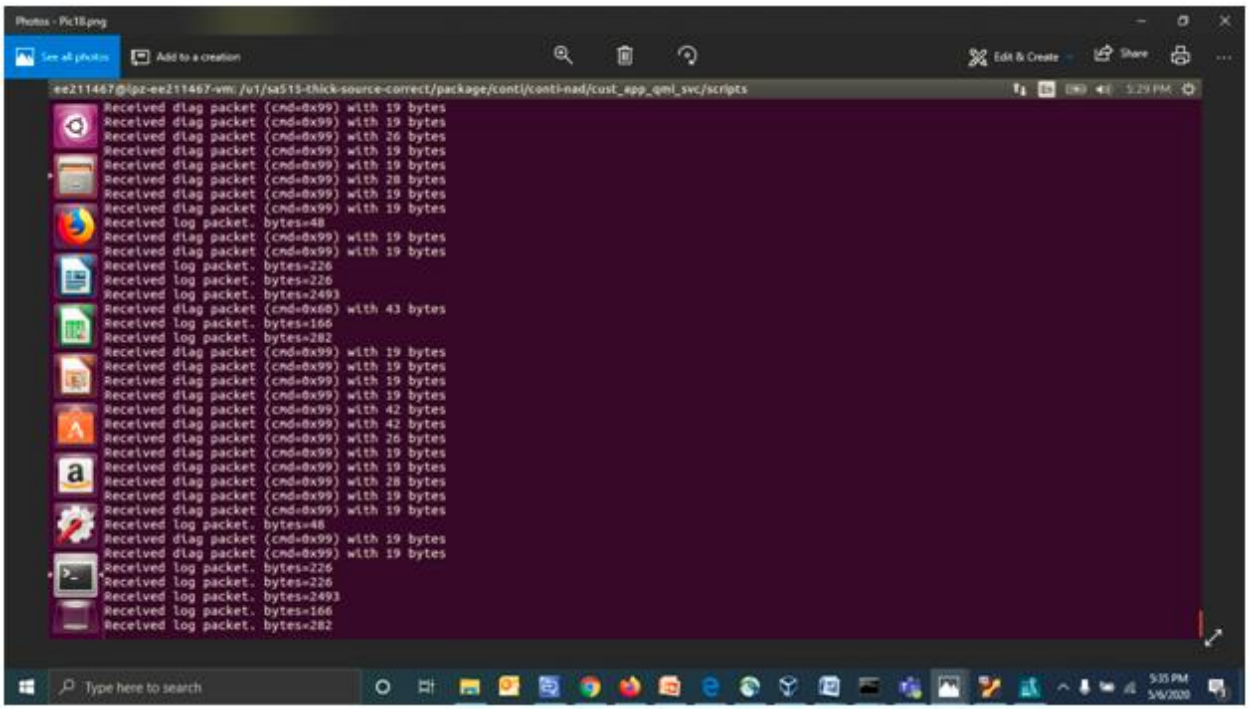


ii. Execute command "ifconfig eth0 *<server IP>* up" from terminal of virtual machine, where *<server IP>* is the IP address of the PC (Windows) Ethernet adapter (screenshots show command using a value of 192.168.225.4).

iii. Ping the target from virtual machine and vice versa.



© Continental Automotive GmbH 2019
Confidential and Proprietary



Note:- Above is a screenshot of running diag_server_set_mask_example.pl in step 1 when the log packets are received at server. The TCP server script saves the log packets received into a single file. This could be modified to store logs into multiple files when size exceeds a limit and remove the older files when the number of files exceeds the storage capacity.

3.7.3.3 Automated logging setup

Section describes usage of automation tool that has been developed to reduce manual configuration and setup during QXDM log collection. Device logging role will be set as client. Logger will work only in server mode.

Tool provides 3 usage options:

- QXDM log collection via ADB/USB
- QXDM log collection via Ethernet (optional)
- Disable logging on device (to be used at the end of logging session)

File list:

- target_logging_setup.py – main automation script
- diag_logger.py – logger server, collects data sent by device
- qxdm_logging.ini – configuration file, used by both scripts above

Tool setup instructions:

1) Python

Make sure you have all Python modules installed and scripts are healthy. Execute:

python -m py_compile target_logging_setup.py

python -m py_compile diag_logger.py

If no errors occurred procced to next section, otherwise install necessary modules

2) ADB

© Continental Automotive GmbH 2019
Confidential and Proprietary

Tool requires ADB version >1.0.31.

Add proper version location to PATH or specify its location in qxdm_logging.ini.

3) Adjusting config file

Config file is qxdm_logging.ini. Should be stored in same location as scripts.

Config description:

[board_config] section:

ip4 - IPv4 Address assigned to eth0 on board. Used only with "eth" option;

ip6 - IPv6 Address assigned to eth0 on board. Used only with "eth" option;

port - port number will be used on device and host. Should be available on both of them;

adb_path - ADB binary that will be used for interaction with device;

mask_source - source to look for mask file. Must be set to “host” or “device” ;

mask - logging mask to be applied. If empty default mask installed on device(/data/on_device_logging/on_device_logging_mask.cfg) will be used.

device_id - device ID that is used by ADB. Could be obtained by execution "adb devices". Recommended to be set. Could be empty if test scenario requires only one ADB device connected to PC. If test scenario requires more than one ADB device or any other device expected to be connected during logging session, this parameter **MUST BE SPECIFIED**

[host_config] section:

ip4 - IPv4 Address assigned to Ethernet interface on host. Used only with "eth" option;

ip6 - IPv6 Address assigned to Ethernet interface on host. Used only with "eth" option;

log_directory - target directory to store log files. Could be empty (logs will be stored in main script directory) or contain absolute/relative path

log_prefix - prefix for log names. If set to "New_log" collected logs will be named as New_log _2025-03-24_16-52-09.qmdl;

max_log_size - max size for log part in MBytes. Complete log will be splitted into sections of this size. If collection stopped by any reason logger will create new file with new timestamp, not continue to update existing one;

Usage:

Tool will look up for device and perform logging configuration. Then the logging server will be run in background.

Soon you will be able to see that logs collection will be started in location of target_logging_setup.py.

Tool can detect device disconnection. In case of physical disconnection or NAD reboot tool will wait till reconnection and restart logging server.

To stop logging collection terminate running script (Ctrl+C). Tool will terminate the server itself.

Using ADB/USB interface:

Connect device to PC using USB and execute:

python target_logging_setup.py adb

Usage Ethernet interface (optional, if physical Ethernet port is available):

Connect device to PC using USB and Ethernet and execute:

python target_logging_setup.py eth

At the end of logging session disable logging on device. To do this execute:

python target_logging_setup.py stop

© Continental Automotive GmbH 2019
Confidential and Proprietary

To check additional options (help) execute:

python target_logging_setup.py help

3.8 JTAG unlock

3.8.1 Overview

3.8.1.1 Platform Content Overview

JTAG is a dedicated debug port implementing a serial communications interface for low-overhead access to the system address and data buses of the SoC. The JTAG interface connects to an on-chip test access port (TAP) that implements a stateful protocol to access a set of test registers that present chip logic levels and device capabilities of various parts.

3.8.1.2 Product Expectations

The JTAG port is used for debugging field return modules. Access to the JTAG port is controlled via blowable fuses. When secure boot is enabled, OEMs must blow the debug disable fuses to prevent access to the processors of the device through the JTAG ports. This action is essential to ensure that secure boot cannot be bypassed.

Once secure boot is enabled, temporary access to JTAG on secure boot enabled devices is possible via debug policy images (`apdp.mbn`).

3.8.1.3 Extending Platform Functionality

Not Applicable

3.8.2 Setup

3.8.2.1 Tools/Equipment Needed

- Personal Computer (PC) with Windows 10 operating system
- winusb USB Host Driver (native on Windows 10)
- High Speed USB cable

3.8.2.2 Setup instructions

Not Applicable

3.8.2.3 Setup Troubleshooting

Not Applicable

3.8.3 Usage

3.8.3.1 Basic Instructions

Not Applicable

3.8.3.2 Detailed Instructions

To unlock JTAG on secure boot enabled devices, flash an appropriately configured debug policy image on to the target.

Update a debug policy configuration file *`/.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml`* by:

1. Set `DP_ENABLE_JTAG` flag to 1.

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
<flag> <bit_pos>2</bit_pos> <value>1</value> </flag> <!--DP_ENABLE_JTAG-->
```

`DP_ENABLE_JTAG` flag:

- 0 (default): JTAG disabled on secure target
- 1: JTAG enabled on secure target

2. Add a device serial number to the list:

```
<serial_num_list> <!--Range: 1 to 200-->
    <serial_num>0x2466e4ca</serial_num>
</serial_num_list>
```

It is possible to specify up to 200 unique serial numbers in this list field. All of the corresponding devices are then supported by the debug policy and subsystem ramdump will work.

3. Make debug policy image `apdp.mbn` (see section "Re-enabling of ram dump, JTAG and Trusted logs" of 5G NAD SecureBoot User Guide - TP_F101U-00050)

4. Add following line to the file `/release/images/devel/4K/rawprogram_nand_p4K_b256K.xml`:

<program PAGES_PER_BLOCK="64" SECTOR_SIZE_IN_BYTES="4096" filename="apdp.mbn" label="apdp" last_sector="7423" num_partition_sectors="128" physical_partition_number="0" start_sector="7296"/>

If the line is already in the file, modify it by setting `apdp.mbn` to the filename field.

5. Enable EDL mode on the target and flash the image from 4K folder using the supplier flashing tools.

3.8.3.3 Testing/Error Reporting

Not Applicable

4 Features/Components – Application Development

This section details internal software interfaces for interacting with the Telematics Platform base software APIs. This is intended for on-target application developers using the Continental Software Development Kit (SDK) for development on the NAD module application processor.

4.1 UART

4.1.1 Overview

4.1.1.1 Platform Content Overview

Access to the UARTs is provided via standard Linux terminal devices. Additionally, the platform provides general purposed APIs that can be used for UART access.

4.1.1.2 Product Expectations

The following expectations must be met by products (users of the platform) for this feature to work as designed:

- Use the platform provided UART access APIs or direct Linux device APIs to open a UART connection
- Use the platform provided UART access APIs or direct Linux device APIs to read UART received data on an open UART
- Use the platform provided UART access APIs or direct Linux device APIs to write data to an open UART

© Continental Automotive GmbH 2019
Confidential and Proprietary

- Use the platform provided UART access APIs or direct Linux device APIs to close a UART connection

4.1.1.3 Extending Platform Functionality

Not Applicable

4.1.2 Setup

4.1.2.1 Tools/Equipment Needed

Not Applicable

4.1.2.2 Setup Instructions

Not Applicable

4.1.2.3 Setup Troubleshooting

Not Applicable

4.1.3 Usage

4.1.3.1 Basic Instructions

Use `tpHAL_LowLevel_OpenUart` to open a UART device for read/write access. Use `tpHAL_LowLevel_ConfigUart` to reconfigure a UART device that is already open. Use `tpHAL_LowLevel_CloseUart` to close a previously opened UART.

When a UART device is open, then `tpHAL_LowLevel_ReadUart` can be used to read data received on the UART and `tpHAL_LowLevel_WriteUart` can be used to write data to be output by the UART.

4.1.3.2 Detailed Instructions

This section contains UART interface APIs and types relating to UART access and configuration. More details can be found in Doxygen with the following format unless stated otherwise:

```
Doxygen Path
    |-> Name of Data Type or API
```

Where *Doxygen Path* is

```
Modules
   |-> Hardware Abstraction Layer (HAL)
      |-> HAL Library
         |-> Universal Asynchronous Receiver/Transmitter (UART)
             Interface
```

API/Type Description	API/Type Name as Referenced in Doxygen Path
Enumeration of HAL API return codes	**Modules** **|-> Hardware Abstraction Layer (HAL)** **|-> HAL Library** **|-> tpHAL_return_t**
UART device configuration structure	**tpHAL_UartConfig_t**
API to open an UART device	**tpHAL_LowLevel_OpenUart**
API to reconfigure an open UART device	**tpHAL_LowLevel_ConfigUart**

© Continental Automotive GmbH 2019
Confidential and Proprietary

API/Type Description	API/Type Name as Referenced in Doxygen Path
API to read data from UART device	**tpHAL_LowLevel_ReadUart**
API to write data from UART device	**tpHAL_LowLevel_WriteUart**
API to close an UART device	**tpHAL_LowLevel_CloseUart**

UART Device configuration

The following UART device names are configured within platform baseline:

IPC UART: /dev/ttyHS0

NAD debug UART: /dev/ttyMSM0

BT UART: /dev/ttyHS1

HOWTO Open an UART device

Use the `tpLowLevel_OpenUart` API to open an UART device. The UART device name (e.g., /dev/ttyHS0) is provided as a text string when the API is invoked. The desired configuration (e.g., baud rate, data bits, stop bits) is also provided in a configuration structure of type `tpHAL_UartConfig_t` when the API is invoked. When the UART device is opened, a UART descriptor is returned that is used to reference the UART device.

HOWTO Reconfigure an UART device

Use the `tpLowLevel_ConfigUart` API to reconfigure an open UART device. The UART descriptor is provided to reference the open UART device when the API is invoked. The new configuration (e.g., baud rate, data bits, stop bits) is also provided in a configuration structure of type `tpHAL_UartConfig_t` when the API is invoked. The new settings provided in the configuration structure are applied to the specified UART device.

HOWTO Read data from an UART device

Use the `tpLowLevel_ReadUart` API to read data from an open UART device. The UART descriptor is provided to reference the open UART device when the API is invoked. A buffer to store the data read from the UART needs to be provided with maximum length of data to be read (e.g. buffer size) when the API is invoked. The read data will be returned in the provided buffer and the length of data read.

HOWTO Write data to an UART device

Use the `tpLowLevel_WriteUart` API to write data to an open UART device. The UART descriptor is provided to reference the open UART device when the API is invoked. A buffer with the data to be written to the UART needs to be provided with the buffer size (e.g., length of data to write) when the API is invoked. The actual number of bytes written to the UART will be returned.

HOWTO Close an UART device

Use the `tpLowLevel_CloseUart` API to close an open UART device. The UART descriptor is provided to reference the open UART device when the API is invoked. The referenced UART device is closed, `tpLowLevel_OpenUart` API must be used to reopen the device for read/write access with a new UART descriptor.

4.1.3.3 Testing/Error Reporting

The UART access APIs will return E_HAL_RET_SUCCESS when the request is successful. The UART access APIs will return commonly used Linux ‘errno’ error codes when a failure occurs. See http://man7.org/linux/man-pages/man3/errno.3.html for more information on 'errno' errors.

© Continental Automotive GmbH 2019
Confidential and Proprietary

4.2 Log & Trace

4.2.1 Logging: Application usage

4.2.1.1 Overview

4.2.1.1.1 Platform Content Overview

Telematics Platform provides a standard way to log information within separate applications using the syslog standard. Applications generate different logs at runtime that can be used to verify the correct functionality or debugging assistance when troubleshooting an error. The collection of all applications messages can be stored in different places: a log file inside the file system, sent over a network or a wired connection, or even RAM.

Messages are grouped by facilities already defined by the standard. These facilities represent subsystems generating messages and this simplifies their routing to specific locations. Telematics Platform has reserved four general purpose facilities (also referred internally as “domains”), while another four are available for application development teams. Next table shows the local facilities and its correspondent domain.

Syslog facility	Telematics Platform domain	Responsible project
`local0`	`TP_LOG_DOMAIN_PRODUCT`	Application development
`local1`	`TP_LOG_DOMAIN_PD_FLEX0`	Application development
`local2`	`TP_LOG_DOMAIN_PD_FLEX1`	Application development
`local3`	`TP_LOG_DOMAIN_PD_FLEX2`	Application development
`local4`	`TP_LOG_DOMAIN_PLF_RESERVE0`	Telematics Platform
`local5`	`TP_LOG_DOMAIN_PLF_RESERVE1`	Telematics Platform
`local6`	`TP_LOG_DOMAIN_PLF_RESERVE2`	Telematics Platform
`local7`	`TP_LOG_DOMAIN_PLATFORM`	Telematics Platform

Message logging includes the concept of message priority/severity levels, ranging from major system impact notices (e.g., fatal errors) to specific operation details (e.g., verbose information). Each log message is assigned a priority/severity level by the developer during code development.

Message log levels can be set on application basis, with increasing message traffic as log level is increased. Thus, using high log levels (e.g., verbose messaging) may impact performance including execution delays. As such, it is recommended that developers consider an appropriate message priority/severity level when defining log messages (i.e., don’t assign every message an “error” level).

Telematics Platform also provides support for stack traces for debugging purposes. For more information about this, please refer to section “Miscellaneous interfaces for debugging” in Common Library User Guide (TP_F101U-00023).

4.2.1.1.2 Product Expectations

The following expectations must be met by products (users of the platform) for this feature to work as designed:

1) The proper domain (syslog facility) is selected when opening a connection with the system logger with `tpLOG_open()`.

   Note: `/data` partition should be available before calling `tpLOG_open()` API.

   Note: Although a library can still log messages, these will appear in the context of the calling application. There’s no need and is recommended to never call `tpLOG_open()` directly from a library.

© Continental Automotive GmbH 2019
Confidential and Proprietary

2) Each application prioritizes the messages to be logged.

3) The proper log level is selected depending on the verbosity level required.

4.2.1.1.3 Extending Platform Functionality

Not Applicable

4.2.1.2 Setup

4.2.1.2.1 Tools/Equipment Needed

Not Applicable

4.2.1.2.2 Setup Instructions

Not Applicable

4.2.1.2.3 Setup Troubleshooting

Not Applicable

4.2.1.3 Usage

4.2.1.3.1 Basic Instructions

Not Applicable

4.2.1.3.2 Detailed Instructions

This section contains Log & Trace interface APIs and types. More details can be found in Doxygen with the following format:

Doxygen Path

|-> ***Name of Data Type or API***

Where *Doxygen Path* is

```
Modules
   |-> Log And Trace
      |-> Logging (LOG)
```

API/Type Description	API/Type Name as Referenced in Doxygen Path
Logging severity level	**tpLOG_severityLevel_t**
API to open a connection with the system logger	**tpLOG_open**
API to log a fatal error into the system logger	**tpLOG_fatal**
API to log a non-fatal error into the system logger	**tpLOG_error**
API to log a warning into the system logger	**tpLOG_warn**
API to log an informative message into the system logger	**tpLOG_info**
API to log a debug message into the system logger	**tpLOG_debug**
API to log a verbose debug message into the system logger	**tpLOG_verbose**

© Continental Automotive GmbH 2019
Confidential and Proprietary

API/Type Description	API/Type Name as Referenced in Doxygen Path
API to log an “Out of Memory” message into the system logger	**tpLOG_oom**
API to log an OS error message into the system logger	**tpLOG_osErr**
API to close the connection with the system logger	**tpLOG_close**
API to set the log level within a specific application	**tpLOG_setLogLevel**
API to get the current log level within a specific application	**tpLOG_getLogLevel**

For an application to start logging messages into the system log, a connection is necessary between it and syslog, specifying the application name and the domain where the messages will be logged. The system then will log only the necessary messages depending on the log level. All APIs in the form `tpLOG_*()` are wrappers of the different log severities defined by syslog.

> **Note:** `tpLOG_verbose()` API will log a “debug” message. `tpLOG_oom()` and `tpLOG_osErr()` requires that log level is at least `E_LOG_SEV_ERROR` to be displayed.

The log level for the specific application can be retrieved with the `tpLOG_getLogLevel()` API and changed with `tpLOG_setLogLevel()` API. The log level is persisted across boots in a very simple way for development environments only.

> **Note:** *Application teams are expected to persist this log level in a secure and reliable manner. Telematics Platform does not guarantee this solution is robust for production environments.* `tpLOG_open()` *will try to read the last persisted value in order to set it correctly.*

How to log application messages into the system logger

The next example shows a very simple application logging different traces depending on severities.

```
#include “tplog.h”
#include <glib.h>
#include <stdlib.h>
#include <stdio.h>

#define APP_NAME “logging-app”
#define BIG_RECOVERY_BUFFER 2048

void dummyCheck(void);
void criticalCheck(void);

int main(int argc, char** argv) {
      tpLOG_open(APP_NAME, TP_LOG_DOMAIN_PRODUCT);

      dummyCheck();
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
	criticalCheck();

	tpLOG_close();

	return EXIT_SUCCESS;
}

void dummyCheck() {
	tpLOG_verbose("Starting dummy check ...");

	FILE* fp = fopen("/data/unknown", "r");

	if (fp == NULL) {
		tpLOG_osErr("fopen", "/data/unknown");
	} else {
		tpLOG_debug("Doing some important actions");
		tpLOG_info("Dummy check success!")
		fclose(fp);
	}
}

void criticalCheck() {
	tpLOG_verbose("Performing critical check on application.");
	guint error_count = 0;

	tpLOG_debug("Creating a buffer to save critical data");
	gchar* buffer = (gchar *) malloc(BIG_RECOVERY_BUFFER);

	if (buffer == NULL) {
		tpLOG_oom();
	}

	// Doing important operations...
	if (error_count > 0) {
		tpLOG_fatal("Critical check failed with %d errors", error_count);
	} else {
		tpLOG_info("Critical check success!");
	}

	free(buffer);
}
```

The result of running a simple program like this is shown below with the different log severities:

© Continental Automotive GmbH 2019
Confidential and Proprietary

E_LOG_SEV_FATAL

E_LOG_SEV_ERROR

```
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
```

E_LOG_SEV_INFO

```
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
local0.info logging-app[7683]: main:Current log level is: E_LOG_SEV_INFO
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
```

E_LOG_SEV_DEBUG

```
local0.info logging-app[7683]: main:Current log level is: E_LOG_SEV_DEBUG
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
local0.debug logging-app[7683]: criticalCheck:Creating a buffer to save critical
data
```

E_LOG_SEV_VERBOSE

```
local0.info logging-app[7683]: main:Current log level is: E_LOG_SEV_VERBOSE
local0.debug logging-app[7683]: dummyCheck:Starting dummy check ...
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
local0.debug logging-app[7683]: criticalCheck:Performing critical check on the
application.
Local0.debug logging-app[7683]: criticalCheck:Creating a buffer to save critical
data
```

How to parse application logs

All application logs have the following format:

```
May  3 03:47:14 localhost local0.debug logging-app[991]: main:Goodbye cruel world!
```

Where:

`May 3 03:47:14`	Current timestamp of when the trace was logged, based on the local time.
`Localhost`	Name of the host (this is useful when having remote servers to log information).
`Local0`	Domain to log, in this case: `TP_LOG_DOMAIN_PRODUCT`.
`Debug`	Severity of the message.
`Logging-app`	Name of the application. This name matches the identifier used in `tpLOG_open()`.
`991`	Process ID of the application.
`Main`	Name of the function.
`Goodbye cruel world!`	Actual message logged into the system.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Having this format in mind, it's possible to apply a regex filtering to display only the required amount of logs: filter by application name, domain, severity, etc.

How to parse kernel messages

These message are intermixed with the application messages on the same `messages` file. They provide useful information about drivers and other operations related to the Linux kernel. These are not expected to show in chronological order because of the nature of its operations.

```
[  352.538001] PM: suspend entry 2018-05-16 01:02:16.569412158 UTC
[  352.544162] PM: Syncing filesystems ... done.
[  352.586617] Freezing user space processes ... (elapsed 0.004 seconds) done.
[  352.596534] Freezing remaining freezable tasks ... (elapsed 0.001 seconds) done.
```

The main difference that can be seen is that the timestamp is a positive value in seconds since the kernel start up followed by the specific function and the actual message. `Dmesg` utility can be used to show kernel ring buffer messages into the console.

4.2.1.3.3 Testing/Error Reporting

Not Applicable

4.2.2 Logging: Configuring and routing log generation

4.2.2.1 Overview

4.2.2.1.1 Platform Content Overview

Telematics Platform provides means to configure the way the logs are stored in the file system. These configurations affect the amount of data being written in memory. Application teams should be aware of the constraints on the total amount of memory allocation in their hardware at the moment of designing the solution.

Also, syslog provides a configuration file that can be used to route specific logs into different locations, depending on the severity and the domain they belong. By default, Telematics Platform routes all messages into `/data/log/messages*` with the exception of all fatal logs generated by the Platform and critical kernel logs, which are stored in `/data/log/critical`.

> **Note:** `/var/log` path is a direct link to `/data/log`, so accessing both results in reading/writing the same log files.

Real files is not the only way to store the generated logs, but these can be send to a remote server (as long as this host is reachable) or in RAM, via virtual files.

4.2.2.1.2 Product Expectations

The following expectations must be met by products (users of the platform) for this feature to work as designed:

1) Platform is configured properly (see Extending Platform Functionality).

2) Application logs are routed properly into the desired location inside `syslog.conf` file.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3) Application teams should verify the flash memory has enough space (`file_size` x `rotate_count`) of storage for the generated log files. That applies for all files configured inside a rule. See **Error! Reference source not found.** section for memory considerations when designing these rules.

4.2.2.1.3 Extending Platform Functionality

The following describes the platform configuration provided for this feature:

LOG-FILE-SIZE

Configuration Type:	Run-Time (Persistency) Key-Value
Description:	Option to configure the total size in KB of a log file before rotation. Maximum allowed size is 2097152 (2 GB). When this value is 0, only one file is created with no maximum size.
Values:	Integer (size in kilobytes) Platform default: 20480 (20 MB) Integer '0' – no maximum size configured. This value is **NOT** recommended. All logs will be stored on a single file until no memory is available.
Attributes:	- Product able to override at compile time - Run-time changes NOT take effect immediately (system reboot is required or restart of the busybox-syslog service).

LOG-ROTATE-COUNT

Configuration Type:	Run-Time (Persistency) Key-Value
Description:	Option to configure the maximum count of rotated files.
Values:	Integer value of maximum log files to rotate. Besides the default log file with no extension, this is the maximum number of rotated files with the next format: <log_file>.0, <log_file>.1, <log_file>.2, …, <log_file>.n – 1 For example, for a defined `messages` file and this value being 5 there will be six files: `messages`, `messages.0`, `messages.1`, `messages.2`, `messages.3`, `messages.4` Platform default: 5 Integer '0' – feature disabled (only one log file will be created; every time the file size exceeds, it will be recreated).
Attributes:	- Product able to override at compile time - Run-time changes NOT take effect immediately (system reboot is required or restart of the busybox-syslog service).

4.2.2.2 Setup

4.2.2.2.1 Tools/Equipment Needed

Not application

4.2.2.2.2 Setup Instructions

Refer to TP_F101U-00026 for more information on how to update values in Persistence items.

4.2.2.2.3 Setup Troubleshooting

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

4.2.2.3 Usage

4.2.2.3.1 Basic Instructions

Not Applicable

4.2.2.3.2 Detailed Instructions

For all rules located inside `syslog.conf`, the log daemon will route the proper domains and severities to the specific locations.

How to route messages from domains into different locations

Next example shows how to separate the application development domains to be logged in different files (regular or virtual).

```
# syslogd configuration file
#
# Log all Facilities and all Priorities to messages file
#
*.*                                     /data/log/messages
local0.debug                            /dev/shm/debug
local1.*                                /dev/console
```

The first rule will send all messages to `/data/log/messages`, the standard output log files from Telematics Platform.

The second rule will forward all critical traces from the `TP_LOG_DOMAIN_PRODUCT` domain to a shared memory location in RAM. This is useful when needing to increase the performance and save space disk.

The third rule will forward all traces from `TP_LOG_DOMAIN_PD_FLEX0` domain to the system console over UART.

The resulting logs for these files would look something like below. Notice the combination of messages on the default `messages` file, but a filtered output on the customized location:

`/data/log/messages`

```
[...]
May  16 01:11:28 localhost local0.debug log-test[6828]: dummyCheck:Starting dummy check ...
May 16 01:11:28 localhost local6.debug test: Auto-generated message
May  16 01:11:29 localhost local0.debug log-test[6828]: dummyCheck:Doing some important actions
May 16 01:11:30 localhost local0.warn log-test[6828]: dummyCheck:Dummy check failed! Performing critical check.
May 16 01:11:31 localhost local6.debug test: Auto-generated message
May  16 01:11:32 localhost local0.debug log-test[6828]: criticalCheck:Performing critical check on the application.
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
May  16 01:11:32 localhost local0.err log-test[6828]: criticalCheck:Trying to save critical data into the filesystem
May 16 01:11:32 localhost local6.debug test: Auto-generated message
May 16 01:11:33 localhost local6.debug test: Auto-generated message
May  16 01:11:33 localhost local0.err log-test[6828]: criticalCheck:46 No memory
May  16 01:11:34 localhost local0.err log-test[6828]: criticalCheck:fwrite No space left to save data. Failed: 'Success'
May  16 01:11:34 localhost local0.emerg log-test[6828]: criticalCheck:No space left on partition.
[...]
```

/var/shm/debug

```
May 16 01:11:28 localhost local6.debug test: Auto-generated message
May 16 01:11:31 localhost local6.debug test: Auto-generated message
May 16 01:11:34 localhost local6.debug test: Auto-generated message
May 16 01:11:35 localhost local6.debug test: Auto-generated message
May 16 01:11:36 localhost local6.debug test: Auto-generated message
May 16 01:11:36 localhost local6.debug test: Auto-generated message
May 16 01:11:38 localhost local6.debug test: Auto-generated message
```

Important note: A mandatory rule is set on `syslog.conf` file to log fatal logs into `/data/log/critical` and this shouldn't be modified.

```
# (DO NOT CHANGE THIS RULE)
# Always store platform fatal logs and critical
# kernel logs in critical file
# TP_LOG_DOMAIN_PLATFORM = LOG_LOCAL7
#
local7.emerg                        /data/log/critical
kern.crit                           /data/log/critical
```

4.2.2.3.3 Testing/Error Reporting

<u>Why am I running out of flash memory?</u>

When designing rules for the syslog.conf file, it's important to note the behavior caused by the persistence items: All configured files will rotate after the maximum size is reached up to number of configured rotation files. There's no way to configure them separately.

Having many rules means that each independent file will grow over time, using space in the flash memory.

For example, having different files to group logs, with the persistence items set as the default values: `LOG-FILE-SIZE` = 20 MB and `LOG-ROTATE-COUNT` = 5:

```
*.*                                 /data/log/messages
local0.debug                        /data/log/pd/debug
local1.*                            /data/log/pd/messages
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

The space needed for maintaining all rotated files (in different rates, depending on how verbose is an application) would be:

20 MB per file x 6 rotated files x 3 rules = 360 MB of free space on the flash memory.

> **Note:** *Very specific grouping like the second rule may cause problems when analyzing issues without the full context of other applications performing tasks in the system.*

4.3 Secure Watchdog

4.3.1 Overview

4.3.1.1 Platform Content Overview

Secure watchdog provides a way to monitor the health of a TrustZone software. This feature is in charge to verify that TrustZone software, while being alive, is still responding.

Only secure watchdog timer (SWDT) can reset the system

- All other WDTs generate an interrupt on bite
- Subsystem WDT bites are handled by HLOS (subsystem restart)
- HLOS WDT bite is handled by TZ (context dump/reset)

Secure WDT bite then resets the device to the first stage of reset.

4.3.1.2 Product Expectations

Not Applicable

4.3.1.3 Extending Platform Functionality

Not Applicable

4.3.2 Setup

4.3.2.1 Tools/Equipment Needed

Not Applicable

4.3.2.2 Setup Instructions

Not Applicable

4.3.2.3 Setup Troubleshooting

Not Applicable

4.3.3 Usage

4.3.3.1 Basic Instructions

Not Applicable

4.3.3.2 Detailed Instructions

Secure Watchdog Timer has the longest bark and bite timeout in the system, maximum of 32 sec. It is configured and accessible only by TrustZone.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Secure Watchdog Timer enabled in cold boot after TZ enables interrupts. TrustZone software then pet Secure Watchdog Timer on bark.

Watchdog Reset Sequence:

- A processor is unable to pet its watchdog. The watchdog expires and bites and notifies the HLOS
- The HLOS cannot pet its watchdog. The HLOS watchdog expires and bites and notifies TZ
- TZ cannot pet its watchdog. The AP's secure watchdog expires and bites and causes a chip reset

Reset due to Watchdog Timer bite

This type of reset occurs when TrustZone hangs somewhere and cannot run to pet the watchdog, because the secure watchdog is pet only by TrustZone.

There could be several reasons that contribute to this type of reset, for example, a core or multiple cores could fail to wake up from power collapse because the wakeup source interrupt was not delivered to the core or TrustZone hung at some point in the warm boot path.

For this case, it is recommended to start with checking TrustZone counters from the IMEM_A dump file to see if the counters match. If warm boot counters or power collapse termination counters match, there was no issue in TrustZone during warm boot or power collapse respectively.

The information needed for debugging includes RAM dump files and the TrustZone symbol file `Tz.elf`, along with other system symbol files.

NOTES:

If the whole NAD module became unresponsive and the external AP doesn't receive any feedback, it is possible to RESET_N power management related GPIO used to reset the module.

Please refer to sections 3.3 and 3.7 of the "*TP_F101U-00039_User_Guide_5G_NAD_Tier1_Power_Management*" for detailed instructions

4.3.3.3 Testing/Error Reporting

Not Applicable

4.4 Process Level Software Watchdog

4.4.1 Overview

4.4.1.1 Platform Content Overview

systemd provides a way to monitor the health of an application that is currently running as a service. This feature is in charge to verify that an application, while being alive, is still responding.

There are some systemd configurations that enable this feature and its main purpose is to perform safe recovery actions individually to those applications that are frozen or locked. Every application is responsible to notify to systemd's service manager the watchdog pings in regular intervals as per its own configuration.

© Continental Automotive GmbH 2019
Confidential and Proprietary

As soon as an application stops issuing the watchdog pings to systemd within the time limit established, the service manager will place the service in a failed state and terminates the application with a `SIGTERM` signal.

This watchdog covers the entire process, instead of an individual thread.

4.4.1.2 Product Expectations

The following expectations must be met by products (users of the platform) in order for this feature to work as designed:

- Application service file contains the proper watchdog timeout and the recovery action to perform.
- On the application source code, read the *`WATCHDOG_USEC`* environment variable after start up. This should be available after systemd successfully started the application.
- Application should issue *`sd_notify(0, "WATCHDOG=1")`* calls before the timeout interval read from the environment variable.

Refer to https://www.freedesktop.org/software/systemd/man/systemd.service.html for more information on how to configure a service file.

4.4.1.3 Extending Platform Functionality

Not applicable

4.4.2 Setup

4.4.2.1 Tools/Equipment Needed

Not applicable

4.4.2.2 Setup Instructions

Use *`pkg-config`* in the client application *`Makefile`* to link to the systemd shared library:

```
LDFLAGS += `pkg-config -libs libsystemd`
CFLAGS += `pkg-config -cflags libsystemd`
```

4.4.2.3 Setup Troubleshooting

Not applicable

4.4.3 Usage

4.4.3.1 Basic Instructions

Not applicable

4.4.3.2 Detailed Instructions

Include the needed headers to read an environment variable and issue a watchdog pat in the application source code:

```
#include <stdlib.h>
#include <systemd/sd-daemon.h>
```

Example:

Next is an example of a .service file with the watchdog monitor enabled for a simple service.

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
[Unit]
Description=Foo

[Service]
ExecStart=/usr/sbin/foo-daemon
WatchdogSec=20s
Restart=on-failure
```

From the application perspective, is needed to read the send regular pings to systemd so it can verify the application is still alive:

1. systemd stores the configured `WatchdogSec` value, in microseconds, in the environment variable *`WATCHDOG_USEC`*. This needs to be read by the application and the application then needs to issue the watchdog ping within a configured time interval.
2. When the application stops issuing the watchdog ping, systemd will restart the application if configured in the service file. Setting action on failure to reboot is not recommended.

Note: The term “restart” refers to the actions that systemd performs to stop and start again a specific application, while the term “reboot” refers to the actions to perform a reboot in the application processor.

4.4.3.3 Testing/Error Reporting

Not applicable

5 Known Issues

Not Applicable

6 FAQs

Not Applicable

7 Platform DocumentationTroubleshooting/Tips and Tricks

Not Applicable

8 References

8.1 Abbreviations/Acronyms/Definitions

Not Applicable

8.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F101U-00019: User Guide NADIF
TP_F101U-00023: Common Service User Guide
TP_F101U-00026: Persistence User Guide
TP_F101U-00035: Software Development Kit User Guide

© Continental Automotive GmbH 2019
Confidential and Proprietary

TP_F101U-00039: 5G NAD Tier 1 Power Management User Guide
TP_F101U-00050: 5G NAD SecureBoot User Guide
5G NAD Module spec: TP_LTE_5G_SA515M_NAD_Specification

8.3 Website links

- syslog standard: https://tools.ietf.org/html/rfc5424
- syslog configuration: https://linux.die.net/man/5/syslog.conf
- Android Debug Bridge (ADB): https://developer.android.com/studio/command-line/adb
- Common Internet File System (CIFS) : https://cifs.com/
- Setting up Samba Share on Ubuntu : https://ubuntu.com/tutorials/install-and-configure-samba#1-overview
- Setting up windows 10 share : https://support.microsoft.com/en-us/windows/file-sharing-over-a-network-in-windows-10-b58704b2-f53a-4b82-7bc1-80f9994725bf

© Continental Automotive GmbH 2019
Confidential and Proprietary

9 Revision History

Version	Date	Description
01.00	Nov 25, 2019	Initial release
02.00	Apr 23, 2020	Modification in fastboot flash method
03.00	May 1, 2020	Add Double Banking section
04.00	Jun 9, 2020	Added Modem DiagnosticSocket Logging section
05.00	Jun 23, 2020	Added details in Double Banking section
06.00	Aug 26, 2020	Modification in Modem Diagnostic Socket Logging section
07.00	Sep 21, 2020	Updated power related GPI and USB Host driver information. Added limited operation note to Modem Diagnostic Socket logging.
08.00	Oct 12, 2020	Add Secure Watchdog and Process Level Software Watchdog sections
09.00	Nov 9, 2020	Added routing modem RAM dump to eMMC
10.00	Feb 23, 2021	Added ADB Shell/USB enable/disable
11.00	Apr 7, 2021	Added SSH integration to the system partition Added SSH terminal access lock/unlock
12.00	May 10, 2021	ADB Shell/USB enable/disable modified and changed to Debug interface lock/unlock SSH terminal access lock/unlock moved to Debug interface lock/unlock. Added JTAG unlock.
13.00	Jun 4, 2021	Added flashing support over ethernet
14.00	Jul 28, 2021	Integrate SSH to the test partition
15.00	Sep 30, 2021	Remove product specific information
== ConMod-specific content from here ==		
16.00	Apr 7, 2025	Added Modem Diagnostic Socket Logging automation tool description
17.00	May 21, 2025	Added new parameters description for Modem Diagnostic Socket Logging automation tool description
18.00	Jan 21, 2026	Added new parameters description for Modem Diagnostic Socket Logging automation tool description

© Continental Automotive GmbH 2019
Confidential and Proprietary